

Chapter 6

2D-DMA Controller (2D-DMA)

6.1 Overview

The 2D-DMA controller is a DMA (Direct Memory Access) dedicated to two-dimensional image processing. In addition to all the features of GDMA-AXI, it includes support for macroblock reordering and color space conversion (CSC) to better meet the data transfer requirements from JPEG and PPA. Notably, the 2D-DMA facilitates memory-to-memory transfers, enabling the movement of macroblocks between different segments of memory address space while concurrently performing color space conversion.

6.2 Features

- One AXI master interface
- Data transfer with unaligned starting addresses
- Memory-to-memory, peripheral-to-memory (RX), and memory-to-peripheral (TX) data transfer
- Four memory-to-peripheral channels, and three peripheral-to-memory channels
- Support for PPA and JPEG Codec
- Macroblock reordering
- Color space conversion
- Configurable channel priority and weight

6.3 Architecture

In ESP32-P4, only JPEG Codec and PPA support 2D-DMA. The 2D-DMA controller and CPU data bus have access to the same address space in memory. Figure [6.3-1](#) shows the basic architecture of the 2D-DMA controller.

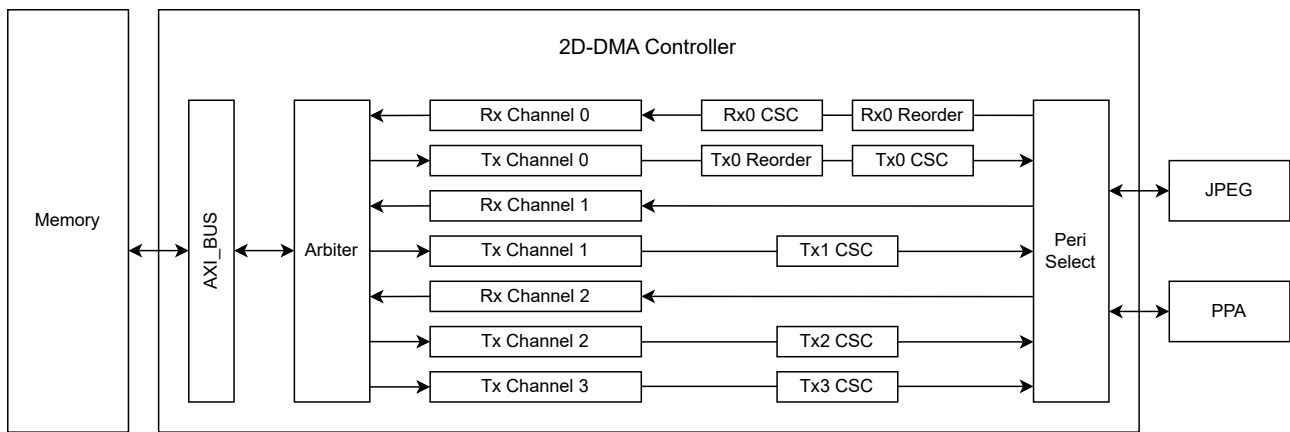


Figure 6.3-1. 2D-DMA Architecture

The 2D-DMA has seven independent channels: four transmit channels and three receive channels. Every channel can be connected to different peripherals, or in other words shared by peripherals.

The macroblock reordering and color space conversion features of the 2D-DMA are not supported by all channels. Among the channels, transmit channel 0 and receive channel 0 support macroblock reordering, while transmit channel 0 ~ 3 and receive channel 0 support color space conversion.

The 2D-DMA reads data from or writes data to the internal or external memory via AXI_BUS. Before the data transfer, the 2D-DMA uses configurable arbitration schemes for channels requesting read or write access. For the available address range of internal and external memory, please see Chapter 7 [System and Memory](#).

The software can use the 2D-DMA through linked lists stored in internal or external memory. These linked lists consist of outlink n and inlink n , where n indicates the channel number. For outlink n , n ranges from 0 to 3; for inlink n , n ranges from 0 to 2. The 2D-DMA reads an outlink n (i.e., a linked list of transmit descriptors) from memory and transmit data in corresponding memory according to the outlink n , or read an inlink n (i.e., a linked list of receive descriptors) and store received data into specific address space in memory according to the inlink n .

6.4 Functional Description

6.4.1 Transfer Mode

The 2D-DMA supports four transfer modes:

- 1D mode: enabled by setting the 2DEN field of DWO and the mod field of DW2 to 0. In this mode, the 2D-DMA has the same functionalities as GDMA-AXI: it performs memory read/write operations based on the data length and memory space configured in the linked list, and the operation addresses are contiguous.
- 2D-MOD0 mode: enabled by setting the 2DEN field of DWO to 1 and the mod field of DW2 to 0. In this mode, the 2D-DMA reads or writes a hb*vb macroblock starting from the (X, Y) coordinates of the HA*VA image. After the last data in each row has been read or written, the address jumps to the address of the first data in the next row to be read or written.

- 2D-MOD1 mode: enabled by setting the 2DEN field of DW0 to 1 and the mod field of DW2 to 1. In this mode, the 2D-DMA reads or writes multiple macroblocks from the starting address of the HA*VA image until the HA*VA image has been fully read or written. For JPEG, it is necessary to configure [DMA2D_IN_MACRO_BLOCK_SIZE_CH_n](#) or [DMA2D_OUT_MACRO_BLOCK_SIZE_CH_n](#) to choose a macroblock size out of the three available options (horizontal × vertical): 8 pixels × 8 pixels, 8 pixels × 16 pixels, and 16 pixels × 16 pixels. For more information about macroblock size, see Chapter [35 JPEG Codec](#). For correspondence between macroblock size, hb, and vb of descriptors, see Table [6.4-4](#) and Table [6.4-5](#).
- DSCR-PORT mode: enabled by setting the 2DEN field of DW0 to 1, the mod field of DW2 to 0, and the [DMA2D_OUT_DSCR_PORT_EN_CH_n](#) bit or the [DMA2D_IN_DSCR_PORT_EN_CH_n](#) bit to 1. This mode is specifically designed for PPA. For details, see Chapter [37 Pixel-Processing Accelerator \(PPA\)](#).

1D Mode, 2D-MOD0 Mode, 2D-MOD1 Mode, and DSCR-PORT support both peripheral-to-memory and memory-to-peripheral data transfers. Only 1D Mode and 2D-MOD0 Mode support memory-to-memory data transfers.

6.4.2 Linked List

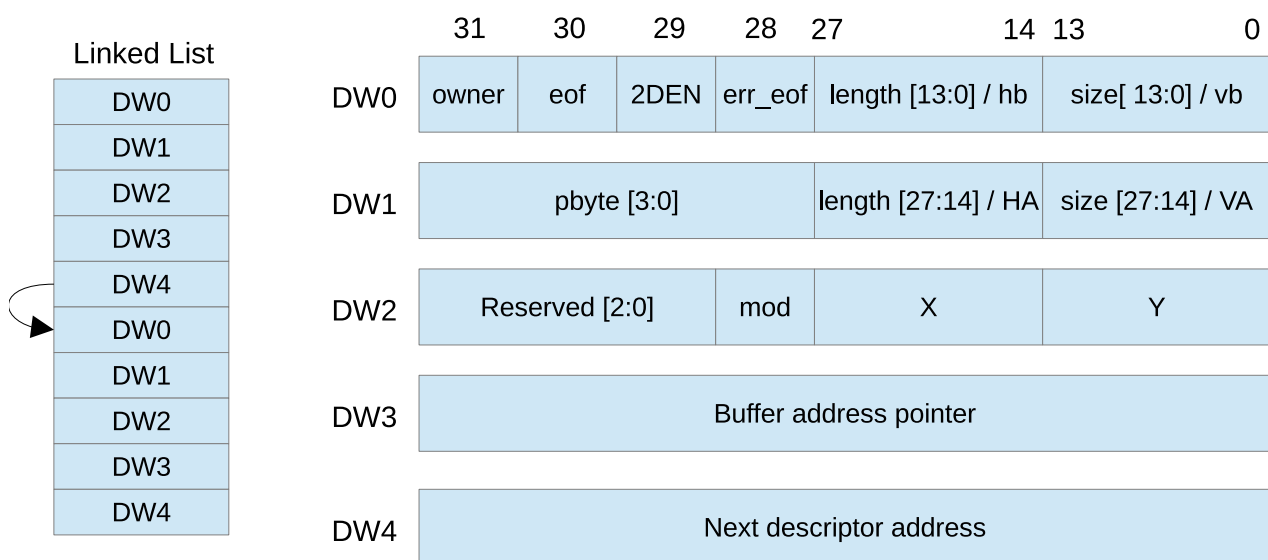


Figure 6.4-1. Structure of a Linked List

Figure [6.4-1](#) shows the structure of a linked list. An outlink and an inlink have the same structure. A linked list is formed by one or more descriptors, and each descriptor consists of five words. Linked lists should be stored in the memory for the 2D-DMA to be able to use them. The meanings of a descriptor's fields are shown in Table [6.4-1](#).

Note: DMA2D_OUT is the prefix of transmit channel registers, and DMA2D_IN is the prefix of receive channel registers.

Table 6.4-1. Descriptor Field Description

Bits	Name	Description
DWO[31]	owner	Specifies who is allowed to access the buffer that this descriptor points to. • 0: CPU can access the buffer • 1: The 2D-DMA controller can access the buffer When the 2D-DMA controller stops using the buffer, this bit in a receive descriptor is automatically cleared by hardware, while this bit in a transmit descriptor can only be automatically cleared by hardware if <code>DMA2D_OUT_AUTO_WRBK_CHn</code> is set to 1. When software loads a linked list, this bit should be set to 1.
DWO[30]	eof	Specifies the end of frame. • 0: The current descriptor is not the last descriptor of an image • 1: The current descriptor is the last descriptor of an image For receive descriptors, software needs to write 0 to this bit first, and hardware would set this bit to 1 after receiving a complete image. For transmit descriptors, software should set this bit in the last descriptor of an image to 1.
DWO[29]	2DEN	Specifies whether to enable 2D functions. • 0: 2D-DMA operates in 1D mode • 1: 2D-DMA operates in 2D mode
DWO[28]	err_eof	Specifies whether the received data has errors (JPEG only). • 0: The received data does not have errors • 1: The received data has errors For receive descriptors, the hardware sets this bit to 1 after receiving a complete image and detecting errors in the received data.
DWO[27:14]	length[13:0]/hb	hb in 2D mode – Specifies the horizontal width of the macroblocks to be moved in the unit of pixels. length[13:0] in 1D mode – Specifies the lower 14 bits of length, which is the number of valid bytes in the buffer that this descriptor points to. This field in a transmit descriptor is written by software and indicates how many bytes can be read from the buffer; this field in a receive descriptor is written by hardware automatically and indicates how many valid bytes have been stored in the buffer.
DWO[13:0]	size[13:0]/vb	vb in 2D mode – Specifies the vertical height of the macroblocks to be moved in the unit of pixels. size[13:0] in 1D mode – Specifies the lower 14 bits of size, which is the capacity of the buffer that this descriptor points to in the unit of bytes.

Bits	Name	Description
DW1[31:28]	pbyte	In 2D mode, specifies the number of bytes per pixel of the image to be transferred to/from the memory. (If color space conversion is applied, this field represents the number of bytes per pixel of the image stored in the memory.) • 0: 0.5 byte/pixel • 1: 1 byte/pixel • 2: 1.5 bytes/pixel • 3: 2 bytes/pixel • 4: 3 bytes/pixel • 5: 4 bytes/pixel In 1D mode, this field is not used and can be any value.
DW1[27:14]	length[27:14]/HA	HA in 2D mode – Specifies the horizontal width of the original image corresponding to the macroblocks to be moved in the unit of pixels. length[27:14] in 1D mode – Specifies the higher 14 bits of length.
DW1[13:0]	size[27:14]/VA	VA in 2D mode – Specifies the vertical height of the original image corresponding to the macroblocks to be moved in the unit of pixels. size[27:14] in 1D mode – Specifies the higher 14 bits of size.
DW2[31:29]	Reserved	Reserved. Irrelevant.
DW2[28]	mod	Specifies the macroblock read mode. • 0: 2D-DMA reads or writes a single macroblock with horizontal width hb and vertical height vb only once • 1: 2D-DMA reads or writes multiple macroblocks with horizontal width hb and vertical height vb until the image data with horizontal width HA and vertical height VA is fully read or written In 1D mode, this bit must be 0. Note: hb/vb/HA/VA are in the unit of pixels.
DW2[27:14]	X	In 2D mode, specifies the X coordinate of the hb*vb image block within the HA*VA image in the unit of pixels. In 1D mode, this field is not used and can be any value.
DW2[13:0]	Y	In 2D mode, specifies the Y coordinate of the hb*vb image block within the HA*VA image in the unit of pixels. In 1D mode, this field is not used and can be any value.
DW3	Buffer address pointer	In 2D mode, specifies the address of the HA*VA image block origin in the buffer (the address of the original image's first byte in the memory). In 1D mode, specifies the starting address of the buffer.

Bits	Name	Description
DW4	Next descriptor address	Address of the next descriptor. If the current descriptor is the last descriptor in the linked list, this value can be 0. This address can point to the address space of internal or external memory. Note: The starting address for storing 2D-DMA descriptors must be 8-byte aligned.

6.4.3 Padding in 2D-MOD1 Mode and DSCR-PORT Mode

This section only applies to the TX direction of 2D-DMA transfers in 2D-MOD1 and DSCR-PORT mode.

For JPEG decoding, the basic unit of image data input is a block of 8×8 , 8×16 , or 16×16 (pixels). In 2D-MOD1 mode, when the length and width of the image are not integer multiples of the basic unit, the image needs to pad out to integer multiples of the basic unit based on the previous or the current macroblock.

For PPA, the basic unit for transmit channels in DSCR-PORT mode is 36×36 (YUV420), 36×34 (YUV422) and 34×34 (horizontal $\times$ vertical). The width and height of a basic unit for different transmit channels are configured using `DMA2D_OUT_DSCR_PORT_BLK_H_CH $n$` and `DMA2D_OUT_DSCR_PORT_BLK_V_CH $n$` . When the length and width of the image are not integer multiples of the basic unit, the image needs to pad out.

Below are the padding rules rules:

- Horizontally, the padding depends on the pbyte field of DW1.
 - When pbyte is 0 (0.5 byte/pixel), the remaining pixels must be even, and the padding is the last 0.5 byte.
 - When pbyte is 1 (1 byte/pixel), the padding is the last 1 byte.
 - When pbyte is 2 (1.5 bytes/pixel), the remaining pixels must be even, and the padding is the last 3 bytes.
 - When pbyte is 3 (2 bytes/pixel), the padding is the last 2 bytes.
 - When pbyte is 4 (3 bytes/pixel), the padding is the last 3 bytes.
 - When pbyte is 5 (4 bytes/pixel), the padding is the last 4 bytes.
- Vertically, the padding is the last row.

Figure 6.4-2 and Figure 6.4-3 illustrate the padding method.

In Figure 6.4-2, a blue grid represents a basic unit (a block of 8×8 , 8×16 , or 16×16 pixels), and a yellow box (incomplete basic unit) along with a green box (padding) forms a basic unit.

In Figure 6.4-3, a grid represents one pixel, and the numbers inside each grid represent pixel values.

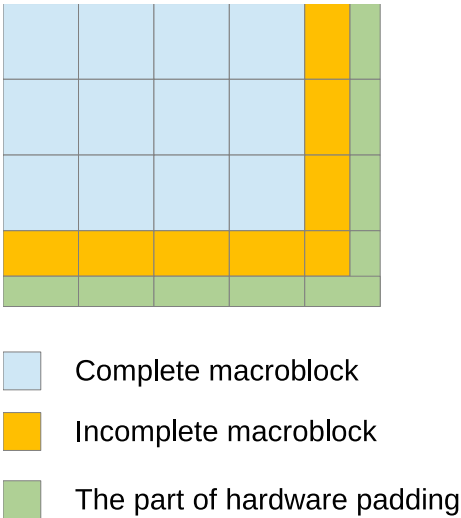


Figure 6.4-2. Padding Illustration (Overall)

1	2	3	3	3	3	3	3
4	5	6	6	6	6	6	6
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9

Figure 6.4-3. Padding Illustration (Detail)

6.4.4 Peripheral-to-Memory and Memory-to-Peripheral Data Transfer

When `DMA2D_MEM_TRANS_EN_CHn` is 0, the 2D-DMA controller enters peripheral transfer mode. The transmit channel reads data from the memory and transmits it to peripherals, while the receive channel receives data from peripherals and stores it into the memory.

The transmit direction refers to data transfer from memory to peripheral, and the receive direction refers data transfer from peripheral to memory. A transmit channel transfers data in the specified memory location to a peripheral’s transmitter via an outlink`n`, whereas a receive channel transfers data received by a peripheral to the specified memory location via an inlink`n`.

Every transmit and receive channel can be connected to any peripheral that supports 2D-DMA. Table 6.4-2 and Table 6.4-3 illustrate how to select the peripheral to be connected via registers. “Dummy-`n`” corresponds to register values for memory-to-memory data transfer. When a channel is connected to a peripheral, the rest

of the channels cannot be connected to that peripheral.

Table 6.4-2. Selecting Peripherals via Register Configuration in TX Direction

DMA2D_PERI_OUT_SEL_CH n	Peripheral
0	JPEG
1	PPA's SRM module
2	PPA's BLEND0 module
3	PPA's BLEND1 module
4 ~ 7	Dummy-4 ~ 7

Table 6.4-3. Selecting Peripherals via Register Configuration in RX Direction

DMA2D_PERI_IN_SEL_CH n	Peripheral
0	JPEG
1	PPA's SRM module
2	PPA's BLEND channel
3 ~ 7	Dummy-3 ~ 7

6.4.5 Memory-to-Memory Data Transfer

The 2D-DMA controller allows memory-to-memory data transfer only in 1D mode and 2D-MOD0 mode. Such data transfer can be enabled by setting `DMA2D_IN_MEM_TRANS_EN_CH $n$` , which connects the output of transmit channel n to the input of receive channel n . Note that a transmit channel can only be connected to the receive channel with the same number (n), and `DMA2D_IN_PERI_SEL_CH $n$` and `DMA2D_OUT_PERI_SEL_CH $n$` should be configured to any value corresponding to "Dummy".

Memory-to-memory data transfer can be used in combination with color space conversion so that a macroblock can be moved from one segment of memory address space to another with its color space converted.

6.4.6 Macroblock Reordering

6.4.6.1 Macroblock Reordering in TX Direction

When the 2D-DMA accesses external memory via SPI, in order to improve the bandwidth utilization of SPI, it is necessary to initiate AXI transfers with a larger burst length. In 2D-MOD1 mode, the 2D-DMA can increase the burst length of AXI transfers by fetching multiple macroblocks at once. For example, the 2D-DMA can fetch five macroblocks of 8x8 pixels in one operation, and in this case, the `hb` field in the descriptor should be 40 and `vb` should be 8. However, the JPEG requires the 2D-DMA to send data in the order of macroblocks. This means that the 2D-DMA needs to reorder the fetched macroblocks to meet the requirements of JPEG.

Recommended configurations for macroblock reordering are listed in Table 6.4-4.

Table 6.4-4. Recommended Configurations for Macroblock Reordering in TX Direction

Byte/Pixel	Macroblock Size (Pixel)	Optimal Macroblock Count	hb	vb
3	8 × 8	5	40	8
3	16 × 8	2	32	8
3	16 × 16	2	32	16
2	8 × 8	8	64	8
2	16 × 8	4	64	8
2	16 × 16	3	48	16
1	8 × 8	16	128	8

The macroblock size in TX direction should be configured via the [DMA2D_OUT_MACRO_BLOCK_SIZE_CHn](#) field.

6.4.6.2 Macroblock Reordering in RX Direction

Currently, in RX direction the 2D-DMA can only reorder the data decoded by JPEG (the bitstream is 1 scan, see Chapter [35 JPEG Codec](#)). The 2D-DMA breaks down color component data decoded by JPEG into pixels, assembles them into larger macroblocks, and writes these blocks into the memory.

The data formats decoded by JPEG include GRAY, YUV444, YUV422, and YUV420. Recommended configurations for macroblock reordering are listed in Table [6.4-5](#).

Table 6.4-5. Recommended Configuration Table for Macroblock Reordering in 2D-DMA Receive Direction (Without Color Format Conversion)

Format	Macroblock Size (pixels)	Optimal Macroblock Count	hb	vb
GRAY	8 × 8	12	96	8
YUV444	8 × 8	5	40	8
YUV422	16 × 8	4	64	8
YUV420	16 × 16	3	48	16

The macroblock size in TX direction should be configured via the [DMA2D_IN_MACRO_BLOCK_SIZE_CHn](#) field.

Figure [6.4-4](#) shows the layout of macroblocks outputted by the JPEG decoder. Each blue grid in this figure represents 8x8 bytes of continuous data, and for each color format the figure displays four macroblocks of 8 × 8 pixels.

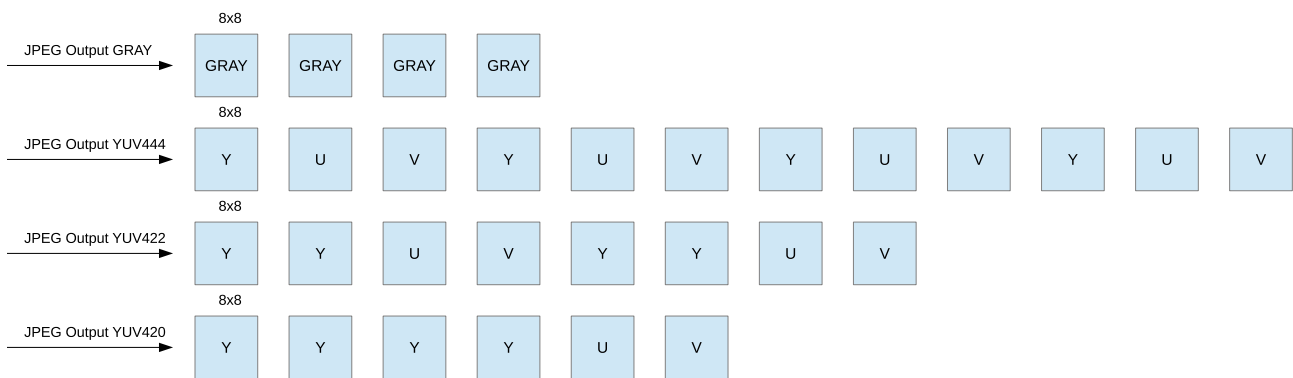


Figure 6.4-4. Macroblock Layout of JPEG Decoder Output Data

6.4.7 Color Space Conversion

On the transmit side, the 2D-DMA converts the color space after assembling the data read from memory into pixels. The 2D-DMA supports the following conversion except YUV420:

- RGB888 to RGB565
- RGB565 to RGB888
- RGB888 to YUV444
- RGB888 to YUV422 (the number of pixels per row in RGB format must be even)
- YUV444 to RGB888
- YUV422 to RGB888

On the receive side, only pixels received from JPEG requires color space conversion. The 2D-DMA supports the following conversion:

- YUV444 to RGB888/RGB565/YUV422/YUV420
- YUV422 to YUV444 to RGB888/RGB565/YUV420
- YUV420 to YUV444 to RGB888/RGB565

Note:

- For JPEG decoding, i.e., the receive side of 2D-DMA, the hardware will forcibly convert YUV422 and YUV420 to the YUV444 format. Software can optionally further convert the format to RGB888/RGB565 with the color space conversion feature.

Figure 6.4-5 shows the structure of the color space conversion module.

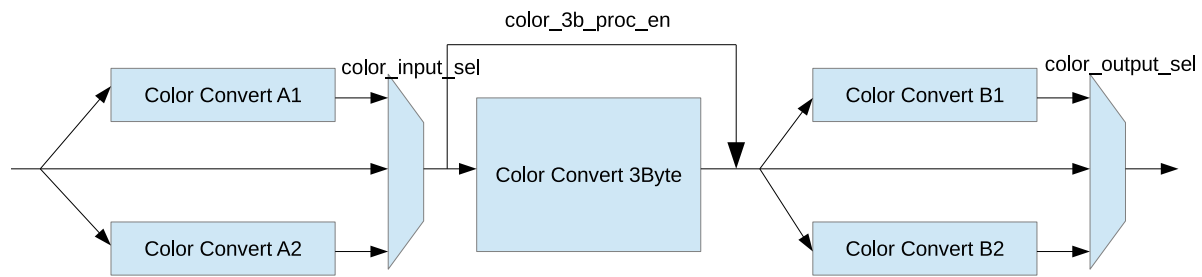


Figure 6.4-5. Color Space Conversion

Color space conversion is performed in 3 stages. The first stage converts a specific 16-bit color format (2 bytes/pixel) to a 24-bit color format (3 bytes/pixel). The source color formats can be configured via the [DMA2D_OUT_COLOR_INPUT_SEL_CHn](#) or [DMA2D_IN_COLOR_INPUT_SEL_CHO](#) field. (See the supported color format conversion in the first stage by clicking the fields)

The second stage further converts the output data of the first stage using the following formula:

$$256 * Q = A[9 : 0] * x + B[10 : 0] * y + C[9 : 0] * z + D[17 : 0]$$

In this formula, Q is the output data; x, y, and z are the most significant, middle, and the least significant byte of the input data respectively; A, B, C, and D are the corresponding multiplication or addition coefficients, which are signed numbers. At most 3 bytes of data can be processed in one go, and therefore $3 * 4 = 12$ coefficients need to be configured via the following fields:

- [DMA2D_OUT_COLOR_PARAM_HO_CHn](#)/[DMA2D_IN_COLOR_PARAM_HO_CHO](#)
- [DMA2D_OUT_COLOR_PARAM_H1_CHn](#)/[DMA2D_IN_COLOR_PARAM_H1_CHO](#)
- [DMA2D_OUT_COLOR_PARAM_MO_CHn](#)/[DMA2D_IN_COLOR_PARAM_MO_CHO](#)
- [DMA2D_OUT_COLOR_PARAM_M1_CHn](#)/[DMA2D_IN_COLOR_PARAM_M1_CHO](#)
- [DMA2D_OUT_COLOR_PARAM_LO_CHn](#)/[DMA2D_IN_COLOR_PARAM_LO_CHO](#)
- [DMA2D_OUT_COLOR_PARAM_L1_CHn](#)/[DMA2D_IN_COLOR_PARAM_L1_CHO](#)

PARAM_Hx, PARAM_Mx, and PARAM_Lx (x is 0 or 1) in field names represent the parameters for the three bytes of data respectively. Take PARAM_Hx as examples, the lower 10 bits of the 21-bit PARAM_H0 is coefficient A, the higher 11 bits of PARAM_H0 is coefficient B, the lower 10 bits of the 28-bit PARAM_H1 is coefficient C, and the higher 18 bits of PARAM_H1 is coefficient D.

The third stage, similar to the first stage, outputs the color format converted in the second stage directly, or further converts the 24-bit color format (3 bytes/pixel) to a 16-bit color format. This is configured using [DMA2D_OUT_COLOR_OUTPUT_SEL_CHn](#) or [DMA2D_IN_COLOR_OUTPUT_SEL_CHO](#).

The following table lists the parameter configurations for commonly used color space conversions:

Table 6.4-6. Parameter Configuration for Color Space Conversion in TX Direction

Conversion	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_l0	param_l1
No conversion	7	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Only scramble order	2/3	0	2	N/A	N/A	N/A	N/A	N/A	N/A

Cont'd on next page

Table 6.4-6 – cont'd from previous page

Conversion	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_IO	param_I1
RGB888 → RGB565	3	0	0	N/A	N/A	N/A	N/A	N/A	N/A
RGB565 → RGB888	0	0	2	N/A	N/A	N/A	N/A	N/A	N/A
RGB888 → (601) YUV444	3	1	2	20442	400019	1EDBDA	2000070	1E8870	20003EE
RGB888 → (709) YUV444	3	1	2	2742F	400010	1EABE6	2000070	1E6870	20003F6
RGB888 → (601) YUV422-MIPI	3	1	1	20442	400019	1EDBDA	2000070	1E8870	20003EE
RGB888 → (709) YUV422-MIPI	3	1	1	2742F	400010	1EABE6	2000070	1E6870	20003F6
YUV444 → (601) RGB888	3	1	2	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV444 → (709) RGB888	3	1	2	12A	C21E5CB	1F252A	1338778	8752A	B7D0800
YUV422-MIPI → (601) RGB888	1	1	2	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV422-MIPI → (709) RGB888	1	1	2	12A	C21E5CB	1F252A	1338778	8752A	B7D0800

Table 6.4-7. Parameter Configuration for Color Space Conversion in RX Direction

Conversion	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_IO	param_I1
No conversion	7	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Only scramble order	1/2	0	1	N/A	N/A	N/A	N/A	N/A	N/A
YUV422 → YUV444	0	0	1	N/A	N/A	N/A	N/A	N/A	N/A
YUV420 → YUV444	0	0	1	N/A	N/A	N/A	N/A	N/A	N/A
YUV444 → YUV422	0	0	2	N/A	N/A	N/A	N/A	N/A	N/A
YUV444 → YUV420	0	0	3	N/A	N/A	N/A	N/A	N/A	N/A
YUV422 → YUV420	0	0	3	N/A	N/A	N/A	N/A	N/A	N/A
YUV420 → (601) RGB888/RGB565	0	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV422 → (601) RGB888/RGB565	0	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV444 → (601) RGB888/RGB565	2	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV420 → (709) RGB888/RGB565	0	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800

Cont'd on next page

Table 6.4-7 – cont'd from previous page

Conversion	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_l0	param_l1
YUV422 → (709) RGB888/RGB565	0	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800
YUV444 → (709) RGB888/RGB565	2	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800

Additionally, the color space conversion module can scramble the order of at most three bytes per pixel. This can be done before the color space conversion in TX direction, and before or after the color space conversion in RX direction. Table 6.4-8, Table 6.4-9, and Table 6.4-10 show the byte order per pixel required by the color space conversion module. For the JPEG decoder, if the byte order per pixel is not scrambled after the macroblock reordering in RX direction, it will conform to the requirements of color space conversion. Assuming that the order of an input pixel is represented from MSB to LSD as 2, 1, 0, the order can be scrambled and outputted as 210 (default), 201, 102, 120, 021, or 012. In RX direction, this feature can be configured before or after the color space conversion via [DMA2D_IN_SCRAMBLE_SEL_PRE_CHO](#) or [DMA2D_IN_SCRAMBLE_SEL_POST_CHO](#) respectively; in TX direction, this feature can be configured before the color space conversion via [DMA2D_OUT_SCRAMBLE_SEL_PRE_CHn](#).

Table 6.4-8. RGB888 Storage Format

	LSB								MSB
LINE	B1	G1	R1	B2	G2	R2	B3	G3	R3

Table 6.4-9. YUV444 Storage Format

	LSB								MSB
LINE	V1	U1	Y1	V2	U2	Y2	V3	U3	Y3

Table 6.4-10. YUV422 Storage Format

	LSB							MSB
LINE	U1	Y1	V1	Y2	U3	Y3	V3	Y4

6.4.8 Enabling 2D-DMA

The software uses the 2D-DMA controller through linked lists. For receive channels, software loads an inlink, configures the [DMA2D_INLINK_ADDR_CHn](#) field with the address of the first receive descriptor, and sets the [DMA2D_INLINK_START_CHn](#) bit to enable the 2D-DMA. For transmit channels, software loads an outlink, prepares data to be transmitted, configures the [DMA2D_OUTLINK_ADDR_CHn](#) field with address of the first transmit descriptor, and sets the [DMA2D_OUTLINK_START_CHn](#) bit to enable the 2D-DMA. The [DMA2D_INLINK_START_CHn](#) bit and [DMA2D_OUTLINK_START_CHn](#) bit are cleared automatically by hardware.

In some cases, you may want to append more descriptors to a 2D-DMA transfer that is already started. Naively, it would seem to be possible to do this by clearing the eof bit of the final descriptor in the existing list and setting its next descriptor address pointer field (DW4) to the first descriptor of the to-be-added list. However, this strategy fails if the existing 2D-DMA transfer is almost or entirely finished. Instead, the 2D-DMA controller

has specialized logic to make sure a transfer can be continued or restarted: if the transfer is ongoing, the controller will make sure to take the appended descriptors into account; if the transfer has already finished, the controller will restart with the new descriptors. This is implemented by the Restart function.

When using the Restart function, software needs to

1. Set [DMA2D_INLINK_STOP_CHn](#) or [DMA2D_OUTLINK_STOP_CHn](#) to stop the 2D-DMA when it finishes processing the current descriptor
2. Rewrite the address of the first descriptor in the new list to DW4 of the last descriptor in the loaded list
3. Set [DMA2D_INLINK_RESTART_CHn](#) or [DMA2D_OUTLINK_RESTART_CHn](#) (these two bits are cleared automatically by hardware)

By doing so, hardware can obtain the address of the first descriptor in the new list when reading the last descriptor in the loaded list, and then read the new list.

6.4.9 Linked List Reading Process

Once configured and enabled by software, the 2D-DMA controller starts to read the linked list from the memory. The 2D-DMA performs checks on descriptors in the linked list. Only if descriptors pass the checks, the corresponding 2D-DMA channel will start data transfer. If the descriptors fail any of the checks, hardware will trigger descriptor error interrupt (either [DMA2D_IN_DSCR_ERR_CHn_INT](#) or [DMA2D_OUT_DSCR_ERR_CHn_INT](#)), and the channel will remain idle. This channel can be restarted by setting [DMA2D_OUTLINK_START_CHn](#) or [DMA2D_INLINK_START_CHn](#) again.

The checks performed on descriptors are:

- Owner bit check when [DMA2D_IN_CHECK_OWNER_CHn](#) or [DMA2D_OUT_CHECK_OWNER_CHn](#) is set to 1. If the owner bit is 0, the buffer is accessed by the CPU. In this case, the owner bit fails the check. The owner bit will not be checked if [DMA2D_IN_CHECK_OWNER_CHn](#) or [DMA2D_OUT_CHECK_OWNER_CHn](#) is 0.
- Descriptor address check, which checks if the descriptor address is located in configured memory space. If a 2D-DMA descriptor points to [DMA2D_ACCESS_INTR_MEM_START_ADDR](#) ~ [DMA2D_ACCESS_INTR_MEM_END_ADDR](#) (the value should be located in internal memory) or [DMA2D_ACCESS_EXTR_MEM_START_ADDR](#) ~ [DMA2D_ACCESS_EXTR_MEM_END_ADDR](#) (the value should be located in external memory), it passes the check. For details, please refer to Section 6.4.11).
- HA/VA/hb/vb/SIZE check, which checks if any of these fields is 0. If any field is 0, it fails the check. HA/VA/hb/vb fields are not checked in 1D mode, while SIZE is checked only in 1D mode.

After the software detects a descriptor error interrupt, it must reset the corresponding channel, and restart the 2D-DMA DMA by setting the [DMA2D_OUTLINK_START_CHn](#) or [DMA2D_INLINK_START_CHn](#) bit.

To quickly obtain receive or transmit descriptors from the memory, AXI burst transfers for 2D-DMA can be enabled by setting [DMA2D_INDSCR_BURST_EN_CHn](#) or [DMA2D_OUTDSCR_BURST_EN_CHn](#).

6.4.10 EOF

Note: In this chapter, EOF of transmit descriptors refers to eof (i.e., bit 30 of DW0), while EOF of receive descriptors refers to both eof and err_eof (i.e., bit 28 of DW0).

The 2D-DMA controller uses EOF (end of frame) flags to indicate the end of the image transmission.

For data transmission, the 2D-DMA generates two types of EOF interrupts:

- `DMA2D_OUT_EOF_CH $n$ _INT`, generated when the eof bit of any descriptor in the linked list is set, and the data corresponding to this descriptor has been transmitted. Usually, the eof bit is set in the last descriptor for the transmission of an image. This interrupt is enabled by setting the `DMA2D_OUT_EOF_CH $n$ _INT_ENA` bit.
- `DMA2D_OUT_TOTAL_EOF_CH $n$ _INT`, generated when the eof bit of the last descriptor in the linked list is set, and the data corresponding to the last descriptor has been transmitted. This interrupt is enabled by setting the `DMA2D_OUT_TOTAL_EOF_CH $n$ _INT_ENA` bit.

For data reception, the 2D-DMA also generates two types of EOF interrupts:

- `DMA2D_IN_SUC_EOF_CH $n$ _INT`, generated when an image has been received. This interrupt is enabled by setting the `DMA2D_IN_SUC_EOF_CH $n$ _INT_ENA` bit.
- (JPEG only) `DMA2D_IN_ERR_EOF_CH $n$ _EOF_INT`, generated when an image has been received with errors. This interrupt is enabled by setting the `DMA2D_IN_ERR_EOF_CH $n$ _INT_ENA` bit, and is valid only when the channel is connected to JPEG.

When detecting an `DMA2D_OUT_TOTAL_EOF_CH $n$ _INT` or `DMA2D_IN_SUC_EOF_CH $n$ _INT` interrupt, software can read the value of the `DMA2D_OUT_EOF_DES_ADDR_CH $n$` or `DMA2D_IN_SUC_EOF_DES_ADDR_CH $n$` field, which stores the address of the finished descriptor. Therefore, the software can tell which descriptors have been used and reclaim them as needed. In RX direction, the address of the descriptor can also be stored to the `DMA2D_IN_ERR_EOF_DES_ADDR_CH $n$` field when an `DMA2D_IN_ERR_EOF_CH $n$ _INT` interrupt is triggered.

6.4.11 Accessing Memory

Any transmit and receive channels of the 2D-DMA can access the internal memory address space configured by `DMA2D_ACCESS_INTR_MEM_START_ADDR` and `DMA2D_ACCESS_INTR_MEM_END_ADDR`, and the external memory address space configured by `DMA2D_ACCESS_EXTR_MEM_START_ADDR` and `DMA2D_ACCESS_EXTR_MEM_END_ADDR`.

The 2D-DMA can send data in the internal memory in burst mode via the AXI bus. Burst transfer on the AXI bus is enabled by default, and cannot be disabled. Users can configure the number of data bytes in a single AXI burst (burst length) to 8, 16, 32, 64, or 128 bytes via `DMA2D_IN_MEM_BURST_LENGTH_CH $n$` and `DMA2D_OUT_MEM_BURST_LENGTH_CH $n$` .

When the 2D-DMA accesses the encrypted external memory space (see Chapter [32 External Memory Encryption and Decryption \(XTS_AES\)](#)), buffer address pointer and corresponding data should be 16-byte aligned; when it accesses the configured internal memory and non-encrypted external memory space, there are no requirements for descriptor field alignment.

6.4.12 Arbitration

To ensure timely response to peripherals running at a high speed with low latency, the 2D-DMA controller implements two arbitration schemes, based on channel priority and channel weight respectively.

- Priority arbitration: Each channel can be assigned a priority from 0 ~15 (in total 16 levels). The larger the number, the higher the priority. When several channels perform data transfers at the same time, the

2D-DMA would respond to the transfer requests according to priority levels, and the channel with higher priority would get a response more timely.

- Weight arbitration:
 - Each channel is assigned a weight (i.e., the number of tokens) from 0 ~ 15. The 2D-DMA divides the AXI bus clock period into multiple time slots, and in each time slot, the number of transfers performed by a channel is determined by the number of its tokens. Every time a channel performs a data transfer, one of its tokens is spent. If all of its tokens have been spent in a time slot, then this channel's transfer request will no longer be responded to until the time slot expires, or until tokens of all channels have been spent and the new time slot starts.
 - If the number of tokens assigned to a channel is not zero, the 2D-DMA waits for this channel's tokens to be spent even if it does not have data transfer requests, and will not exit from this time slot ahead of expiration. This leads to a waste of bandwidth. Therefore, the 2D-DMA provides weight arbitration optimization. When this feature is enabled, a channel without data transfer requests will not be involved in the arbitration and its tokens will be ignored. When all channels no longer have transfer requests, the 2D-DMA arbiter exits from the current time slot before expiration. This way, the arbitration response is accelerated and the transfer efficiency is improved.

Note:

- If channels have the same weight but different priorities, on condition that the number of bytes to be transferred is the same and not large, channels with higher priorities would finish data transfers first.
- If channels have different priorities and weights, on condition that the number of bytes to be transferred is the same, channels with higher weights would be allocated more bandwidth and finish data transfers first.

6.5 Event Task Matrix Feature

The 2D-DMA controller on ESP32-P4 supports the Event Task Matrix (ETM) function, which allows 2D-DMA's ETM tasks to be triggered by any peripherals' ETM events, or 2D-DMA's ETM events to trigger any peripherals' ETM tasks. This section introduces the ETM tasks and events related to 2D-DMA. For more information, please refer to [Chapter 13 Event Task Matrix \(ETM\)](#).

2D-DMA can receive the following ETM tasks:

- `DMA2D_TASK_IN_START_CH $n$` : Enables the corresponding receive channel n for data transfer.
- `DMA2D_TASK_OUT_START_CH $n$` : Enables the corresponding transmit channel n for data transfer.

Note:

Above ETM tasks can achieve the same functions as CPU configuring `DMA2D_INLINK_START_CH $n$` and `DMA2D_OUTLINK_START_CH $n$` . When `DMA2D_IN_ETM_EN_CH $n$` or `DMA2D_OUT_ETM_EN_CH $n$` is 1, only ETM tasks can be used to configure the transfer direction and enable the corresponding 2D-DMA channel. When `DMA2D_IN_ETM_EN_CH $n$` or `DMA2D_OUT_ETM_EN_CH $n$` is 0, only CPU can be used to enable the corresponding 2D-DMA channel.

- `DMA2D_TASK_IN_DSCR_READY_CH $n$` : Allows receive channel n to process the next descriptor.

- `DMA2D_TASK_OUT_DSCR_READY_CH $n$` : Allows transmit channel n to process the next descriptor.

Note:

In normal conditions, after finishing with the current descriptor if the next descriptor is available (i.e., the next descriptor address field is not zero), the 2D-DMA would process the next descriptor directly and reads/writes to the memory according to the descriptor's configuration until all descriptors are processed. When `DMA2D_IN_ETM_LOOP_EN_CH $n$` or `DMA2D_OUT_ETM_LOOP_EN_CH $n$` is set, after finishing with the current descriptor, the 2D-DMA would not start processing the next descriptor until receiving the `DMA2D_TASK_IN_DSCR_READY_CH $n$` or `DMA2D_TASK_OUT_DSCR_READY_CH $n$` task.

The `DMA2D_TASK_IN_DSCR_READY_CH $n$` or `DMA2D_TASK_OUT_DSCR_READY_CH $n$` task can be stored in the task buffer, which can cache at most four tasks. When the 2D-DMA receives this task but the current descriptor is not yet processed, it can cache the task. The depth of the task buffer is determined by `DMA2D_IN_DSCR_TASK_MAX_CH $n$` or `DMA2D_OUT_DSCR_TASK_MAX_CH $n$` .

When the task buffer exceeds its maximum capacity, it will trigger an `DMA2D_IN_DSCR_TASK_OVF_CH $n$ _INT` or `DMA2D_OUT_DSCR_TASK_OVF_CH $n$ _INT` interrupt.

2D-DMA can generate the following ETM events:

- `DMA2D_EVT_IN_DONE_CH $n$` : Indicates that the data has been received according to the receive descriptor via channel n .
- `DMA2D_EVT_IN_SUC_EOF_CH $n$` : Indicates that the data corresponding to a receive descriptor has been received via channel n and the Eof bit of this descriptor is 1.
- `DMA2D_EVT_OUT_DONE_CH $n$` : Indicates that the data has been transmitted according to the transmit descriptor via channel n .
- `DMA2D_EVT_OUT_SUC_EOF_CH $n$` : Indicates that the data corresponding to a transmit descriptor has been transmitted or received via channel n and the eof bit of this descriptor is 1.
- `DMA2D_EVT_OUT_TOTAL_EOF_CH $n$` : Indicates that the data corresponding to the last transmit descriptors has been sent via transmit channel n and the eof bit of this descriptor is 1.

In practical applications, 2D-DMA's ETM events can trigger its own ETM tasks. For example, the `DMA2D_EVT_OUT_TOTAL_EOF_CH0` event can trigger the `DMA2D_TASK_IN_START_CH1` task, and in this way trigger a new round of 2D-DMA operations.

6.6 Interrupts

ESP32-P4's 2D-DMA module can generate the following interrupt signals that will be sent to the [Interrupt Matrix](#).

- `DMA2D_IN_CHO_INTR`
- `DMA2D_IN_CH1_INTR`
- `DMA2D_IN_CH2_INTR`
- `DMA2D_OUT_CHO_INTR`
- `DMA2D_OUT_CH1_INTR`

- DMA2D_OUT_CH2_INTR
- DMA2D_OUT_CH3_INTR

There are several internal interrupt sources from 2D-DMA that can generate the above interrupt signals.

The DMA2D_IN_CH n _INTR (n is 0 ~ 2) signals are generated by the following interrupt sources:

- DMA2D_IN_DONE_CH n _INT: Triggered when all data corresponding to a receive descriptor has been received via receive channel n .
- DMA2D_IN_SUC_EOF_CH n _INT: Triggered when the eof bit in a receive descriptor is 1 and the data corresponding to this receive descriptor has been received (i.e., when an image corresponding to an inlink has been received) via receive channel n .
- DMA2D_IN_ERR_EOF_CH n _INT: Triggered when an error is detected in the image received via receive channel n . This interrupt is used only for JPEG.
- DMA2D_IN_DSCR_EMPTY_CH n _INT: Triggered when the size of the buffer pointed by receive descriptors is smaller than the length of data to be received via receive channel n .
- DMA2D_IN_DSCR_ERR_CH n _INT: Triggered when a receive descriptor on receive channel n fails any of the two descriptor checks.
- DMA2D_INFIFO_OVF_CH n _INT: Triggered when the RX FIFO of 2D-DMA overflows.
- DMA2D_INFIFO_UDF_CH n _INT or AXI_DMA_INFIFO_L1/L2/L3_UDF_CH n _INT: Triggered when the RX FIFO of 2D-DMA underflows.
- DMA2D_IN_DSCR_TASK_OVF_CH n _INT: Triggered when the number of cached DMA2D_TASK_IN_DSCR_READY_CH n task exceeds the maximum capacity of the task buffer.
- DMA2D_INFIFO_RO_OVF_CHO_INT (only channel 0): Triggered when the buffer for macroblock reordering in RX direction overflows.
- DMA2D_INFIFO_RO_UDF_CHO_INT (only channel 0): Triggered when the buffer for macroblock reordering in RX direction underflows.

The DMA2D_OUT_CH n _INTR (n is 0 ~ 3) signals are generated by the following interrupt sources:

- DMA2D_OUT_DONE_CH n _INT: Triggered when all data corresponding to a transmit descriptor has been sent via transmit channel n .
- DMA2D_OUT_EOF_CH n _INT: Triggered when the eof bit in a transmit descriptor is 1 and data corresponding to this descriptor has been sent via transmit channel n . If [DMA2D_OUT_EOF_MODE_CH \$n\$](#) is 0, this interrupt will be triggered when the last byte of data corresponding to this descriptor enters 2D-DMA's transmit channel via the bus; if [DMA2D_OUT_EOF_MODE_CH \$n\$](#) is 1, this interrupt is triggered when the last byte of data is taken from 2D-DMA's transmit channel.
- DMA2D_OUT_DSCR_ERR_CH n _INT: Triggered when a transmit descriptor on transmit channel n fails any of the two descriptor checks.
- DMA2D_OUT_TOTAL_EOF_CH n _INT: Triggered when all data corresponding to a linked list (including multiple descriptors) has been sent via transmit channel n .
- DMA2D_OUTFIFO_L1/L2/L3_OVF_CH n _INT: Triggered when the TX FIFO of 2D-DMA overflows.
- DMA2D_OUTFIFO_L1/L2/L3_UDF_CH n _INT: Triggered when the TX FIFO of 2D-DMA underflows.

- `DMA2D_OUT_DSCR_TASK_OVF_CH $n$ _INT`: Triggered when the number of cached `DMA2D_TASK_OUT_DSCR_READY_CH $n$` task exceeds the maximum capacity of the task buffer.
- `DMA2D_OUTFIFO_RO_OVF_CHO_INT` (only channel 0): Triggered when the buffer for macroblock reordering or color space conversion in TX direction overflows.
- `DMA2D_OUTFIFO_RO_UDF_CHO_INT` (only channel 0): Triggered when the buffer for macroblock reordering or color space conversion in TX direction underflows.

6.7 Programming Procedures

The clock gating for the 2D-DMA can be configured via `HP_SYS_CLKRST_DMA2D_SYS_CLK_EN`, and is enabled by default. The 2D-DMA can be reset globally by configuring `HP_SYS_CLKRST_RST_EN_AHB/AXI_PDMA`.

6.7.1 General Configurations for 2D-DMA

Configure the accessible address space of internal and external memory and burst length according to Section 6.4.11.

6.7.2 Mode-Specific Configurations for 2D-DMA

- 1D Mode: No specific configuration required.
- 2D-MOD0 Mode: No specific configuration required.
- 2D-MOD1 Mode: For JPEG, configure `DMA2D_IN_MACRO_BLOCK_SIZE_CH $n$` or `DMA2D_OUT_MACRO_BLOCK_SIZE_CH $n$` to choose a macroblock size out of the three available options (horizontal $\times$ vertical): 8 pixels $\times$ 8 pixels, 8 pixels $\times$ 16 pixels, and 16 pixels $\times$ 16 pixels. For more information about macroblock size, see Chapter 35 *JPEG Codec*. For correspondence between macroblock size, hb, and vb of descriptors, see Table 6.4-4 and Table 6.4-5.
- DSCR-PORT Mode: Set `DMA2D_OUT_DSCR_PORT_EN_CH $n$` to enter this mode. This mode is specifically designed for PPA. For details, see Chapter 37 *Pixel-Processing Accelerator (PPA)*.

6.7.3 Configurations for 2D-DMA's Transmit Channel

To transmit data, 2D-DMA's transmit channel should be configured by software as follows:

1. Set `DMA2D_OUT_RST_CH $n$` first to 1 and then to 0, to reset the state machine of 2D-DMA's transmit channel and FIFO pointer.
2. Load an outlink, and configure `DMA2D_OUTLINK_ADDR_CH $n$` with address of the first transmit descriptor.
3. Configure `DMA2D_OUT_PERI_SEL_CH $n$` with the value corresponding to the peripheral to be connected, as shown in Table 6.4-2.
4. Set `DMA2D_OUTLINK_START_CH $n$` to enable 2D-DMA's transmit channel for data transfer.
5. Configure and enable the corresponding peripheral. See details in individual chapters of these peripherals.
6. Wait for the `DMA2D_OUT_EOF_CH $n$ _INT` interrupt, which indicates the completion of data transfer.

6.7.4 Configurations for 2D-DMA's Receive Channel

To receive data, 2D-DMA's receive channel should be configured by software as follows:

1. Set `DMA2D_IN_RST_CH $n$` first to 1 and then to 0, to reset the state machine of 2D-DMA's receive channel and FIFO pointer.
2. Load an inlink, and configure `DMA2D_INLINK_ADDR_CH $n$` with address of the first receive descriptor.
3. Configure `DMA2D_IN_PERI_SEL_CH $n$` with the value corresponding to the peripheral to be connected, as shown in Table 6.4-3.
4. Set `DMA2D_INLINK_START_CH $n$` to enable 2D-DMA's receive channel for data transfer.
5. Configure and enable the corresponding peripheral. See details in individual chapters of these peripherals.
6. Wait for `DMA2D_IN_SUC_EOF_CH $n$ _INT` interrupt, which indicates that an image has been received.

6.7.5 Configurations for Memory-to-Memory Transfer

To transfer data from one memory location to another, 2D-DMA should be configured by software as follows:

1. Set `DMA2D_OUT_RST_CH $n$` first to 1 and then to 0, to reset the state machine of 2D-DMA's transmit channel and FIFO pointer.
2. Set `DMA2D_IN_RST_CH $n$` first to 1 and then to 0, to reset the state machine of 2D-DMA's receive channel and FIFO pointer.
3. Load an outlink, and configure `DMA2D_OUTLINK_ADDR_CH $n$` with address of the first transmit descriptor.
4. Load an inlink, and configure `DMA2D_INLINK_ADDR_CH $n$` with address of the first receive descriptor.
5. Configure `DMA2D_PERI_OUT_SEL_CH $n$` with any value corresponding to Dummy- n (see Table 6.4-2).
6. Configure `DMA2D_PERI_IN_SEL_CH $n$` with any value corresponding to Dummy- n (see Table 6.4-3).
7. Set `DMA2D_IN_MEM_TRANS_EN_CH $n$` to enable memory-to-memory transfer.
8. Set `DMA2D_OUTLINK_START_CH $n$` to enable 2D-DMA's transmit channel for data transfer.
9. Set `DMA2D_INLINK_START_CH $n$` to enable 2D-DMA's receive channel for data transfer.
10. Wait for `DMA2D_IN_SUC_EOF_CH $n$ _INT` interrupt, which indicates that a data transaction has been completed.

6.7.6 Configurations for Channel Priority and Weight

The priority arbitration can be configured as follows:

1. Configure the channel priority for TX and RX respectively via `DMA2D_OUT_ARB_PRIORITY_CH $n$` and `DMA2D_IN_ARB_PRIORITY_CH $n$` .

The weight arbitration can be configured as follows:

1. Configure the time slot for TX and RX respectively via `DMA2D_OUT_ARB_TIMEOUT_NUM` and `DMA2D_IN_ARB_TIMEOUT_NUM`.

2. Configure the number of tokens for TX and RX respectively via [DMA2D_OUT_ARB_TOKEN_NUM_CH \$n\$](#) and [DMA2D_IN_ARB_TOKEN_NUM_CH \$n\$](#) .
3. Enable weight arbitration optimization for TX and RX respectively by clearing [DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CH_CH \$n\$](#) and [DMA2D_IN_ARB_WEIGHT_OPT_DIS_CH \$n\$](#) .
4. Enable the arbitration for TX and RX respectively via [DMA2D_OUT_WEIGHT_EN](#) and [DMA2D_IN_WEIGHT_EN](#).

6.7.7 Resetting 2D-DMA While it Runs

When the 2D-DMA is running, namely when it is still processing descriptors, it must be stopped first before being reset fully via [HP_SYS_CLKRST_RST_EN_DMA2D](#) or partially (only reset AXI functions) via [DMA2D_AXIM_RD_RST](#) and [DMA2D_AXIM_WR_RST](#). The procedures are as follows:

1. Set [DMA2D_IN_CMD_DISABLE_CH \$n\$](#) and [DMA2D_OUT_CMD_DISABLE_CH \$n\$](#) to stop the 2D-DMA.
2. Reset the 2D-DMA when [DMA2D_OUT_RESET_AVAIL_CH \$n\$](#) and [DMA2D_IN_RESET_AVAIL_CH \$n\$](#) turn to 1.
3. Clear [DMA2D_IN_CMD_DISABLE_CH \$n\$](#) and [DMA2D_OUT_CMD_DISABLE_CH \$n\$](#) to 0 after the 2D-DMA is reset.

6.8 Register Summary

The addresses in this section are relative to 2D-DMA base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Configuration Registers			
DMA2D_OUT_CONFO_CHO_REG	Configures the TX channel 0	0x0000	R/W
DMA2D_OUT_PUSH_CHO_REG	Configures the FIFO of TX channel 0	0x0018	varies
DMA2D_OUT_LINK_CONF_CHO_REG	Configures the transmit descriptor operations of TX channel 0	0x001C	varies
DMA2D_OUT_LINK_ADDR_CHO_REG	Configures the transmit descriptor address of TX channel 0	0x0020	R/W
DMA2D_OUT_ARB_CHO_REG	Configures the arbitration of TX channel 0	0x003C	R/W
DMA2D_OUT_RO_PD_CONF_CHO_REG	Configures the macroblock reordering memory of TX channel 0	0x0044	R/W
DMA2D_OUT_COLOR_CONVERT_CHO_REG	Configures the color space conversion of TX channel 0	0x0048	R/W
DMA2D_OUT_SCRAMBLE_CHO_REG	Configures the byte scrambling of TX channel 0	0x004C	R/W
DMA2D_OUT_COLOR_PARAM0_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x0050	R/W
DMA2D_OUT_COLOR_PARAM1_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x0054	R/W
DMA2D_OUT_COLOR_PARAM2_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x0058	R/W
DMA2D_OUT_COLOR_PARAM3_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x005C	R/W
DMA2D_OUT_COLOR_PARAM4_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x0060	R/W
DMA2D_OUT_COLOR_PARAM5_CHO_REG	Configures the color space conversion parameter of TX channel 0	0x0064	R/W
DMA2D_OUT_ETM_CONF_CHO_REG	Configures the ETM of TX channel 0	0x0068	R/W
DMA2D_OUT_DSCR_PORT_BLK_CHO_REG	Configures the block size for TX channel 0 in DSCR-PORT mode	0x006C	R/W
DMA2D_OUT_CONFO_CH1_REG	Configures the TX channel 1	0x0100	R/W
DMA2D_OUT_PUSH_CH1_REG	Configures the FIFO of TX channel 1	0x0118	varies
DMA2D_OUT_LINK_CONF_CH1_REG	Configures the transmit descriptor operations of TX channel 1	0x011C	varies
DMA2D_OUT_LINK_ADDR_CH1_REG	Configures the transmit descriptor address of TX channel 1	0x0120	R/W
DMA2D_OUT_ARB_CH1_REG	Configures the arbitration of TX channel 1	0x013C	R/W
DMA2D_OUT_COLOR_CONVERT_CH1_REG	Configures the color space conversion of TX channel 1	0x0148	R/W
DMA2D_OUT_SCRAMBLE_CH1_REG	Configures the byte scrambling of TX channel 1	0x014C	R/W

Name	Description	Address	Access
DMA2D_OUT_COLOR_PARAM0_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x0150	R/W
DMA2D_OUT_COLOR_PARAM1_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x0154	R/W
DMA2D_OUT_COLOR_PARAM2_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x0158	R/W
DMA2D_OUT_COLOR_PARAM3_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x015C	R/W
DMA2D_OUT_COLOR_PARAM4_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x0160	R/W
DMA2D_OUT_COLOR_PARAM5_CH1_REG	Configures the color space conversion parameter of TX channel 1	0x0164	R/W
DMA2D_OUT_ETM_CONF_CH1_REG	Configures the ETM of TX channel 1	0x0168	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH1_REG	Configures the block size for TX channel 1 in DSCR-PORT mode	0x016C	R/W
DMA2D_OUT_CONFO_CH2_REG	Configures the TX channel 2	0x0200	R/W
DMA2D_OUT_PUSH_CH2_REG	Configures the FIFO of TX channel 2	0x0218	varies
DMA2D_OUT_LINK_CONF_CH2_REG	Configures the transmit descriptor operations of TX channel 2	0x021C	varies
DMA2D_OUT_LINK_ADDR_CH2_REG	Configures the transmit descriptor address of TX channel 2	0x0220	R/W
DMA2D_OUT_ARB_CH2_REG	Configures the arbitration of TX channel 2	0x023C	R/W
DMA2D_OUT_COLOR_CONVERT_CH2_REG	Configures the color space conversion of TX channel 2	0x0248	R/W
DMA2D_OUT_SCRAMBLE_CH2_REG	Configures the byte scrambling of TX channel 2	0x024C	R/W
DMA2D_OUT_COLOR_PARAM0_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x0250	R/W
DMA2D_OUT_COLOR_PARAM1_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x0254	R/W
DMA2D_OUT_COLOR_PARAM2_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x0258	R/W
DMA2D_OUT_COLOR_PARAM3_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x025C	R/W
DMA2D_OUT_COLOR_PARAM4_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x0260	R/W
DMA2D_OUT_COLOR_PARAM5_CH2_REG	Configures the color space conversion parameter of TX channel 2	0x0264	R/W
DMA2D_OUT_ETM_CONF_CH2_REG	Configures the ETM of TX channel 2	0x0268	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH2_REG	Configures the block size for TX channel 2 in DSCR-PORT mode	0x026C	R/W
DMA2D_OUT_CONFO_CH3_REG	Configures the TX channel 3	0x0300	R/W
DMA2D_OUT_PUSH_CH3_REG	Configures the FIFO of TX channel 3	0x0318	varies
DMA2D_OUT_LINK_CONF_CH3_REG	Configures the transmit descriptor operations of TX channel 3	0x031C	varies
DMA2D_OUT_LINK_ADDR_CH3_REG	Configures the transmit descriptor address of TX channel 3	0x0320	R/W
DMA2D_OUT_ARB_CH3_REG	Configures the arbitration of TX channel 3	0x033C	R/W
DMA2D_OUT_COLOR_CONVERT_CH3_REG	Configures the color space conversion of TX channel 3	0x0348	R/W

Name	Description	Address	Access
DMA2D_OUT_SCRAMBLE_CH3_REG	Configures the byte scrambling of TX channel 3	0x034C	R/W
DMA2D_OUT_COLOR_PARAM0_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x0350	R/W
DMA2D_OUT_COLOR_PARAM1_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x0354	R/W
DMA2D_OUT_COLOR_PARAM2_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x0358	R/W
DMA2D_OUT_COLOR_PARAM3_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x035C	R/W
DMA2D_OUT_COLOR_PARAM4_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x0360	R/W
DMA2D_OUT_COLOR_PARAM5_CH3_REG	Configures the color space conversion parameter of TX channel 3	0x0364	R/W
DMA2D_OUT_ETM_CONF_CH3_REG	Configures the ETM of TX channel 3	0x0368	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH3_REG	Configures the block size for TX channel 3 in DSCR-PORT mode	0x036C	R/W
DMA2D_IN_CONFO_CHO_REG	Configures the RX channel 0	0x0500	R/W
DMA2D_IN_POP_CHO_REG	Configures the FIFO of RX channel 0	0x0518	varies
DMA2D_IN_LINK_CONF_CHO_REG	Configures the receive descriptor operations of RX channel 0	0x051C	varies
DMA2D_IN_LINK_ADDR_CHO_REG	Configures the receive descriptor address of RX channel 0	0x0520	R/W
DMA2D_IN_ARB_CHO_REG	Configures the arbitration of RX channel 0	0x0540	R/W
DMA2D_IN_RO_PD_CONF_CHO_REG	Configures the macroblock reordering memory of RX channel 0	0x0548	R/W
DMA2D_IN_COLOR_CONVERT_CHO_REG	Configures the color space conversion of RX channel 0	0x054C	R/W
DMA2D_IN_SCRAMBLE_CHO_REG	Configures the byte scrambling of RX channel 0	0x0550	R/W
DMA2D_IN_COLOR_PARAM0_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x0554	R/W
DMA2D_IN_COLOR_PARAM1_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x0558	R/W
DMA2D_IN_COLOR_PARAM2_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x055C	R/W
DMA2D_IN_COLOR_PARAM3_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x0560	R/W
DMA2D_IN_COLOR_PARAM4_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x0564	R/W
DMA2D_IN_COLOR_PARAM5_CHO_REG	Configures the color space conversion parameter of RX channel 0	0x0568	R/W
DMA2D_IN_ETM_CONF_CHO_REG	Configures the ETM of RX channel 0	0x056C	R/W
DMA2D_IN_CONFO_CH1_REG	Configures the RX channel 1	0x0600	R/W
DMA2D_IN_POP_CH1_REG	Configures the FIFO of RX channel 1	0x0618	varies
DMA2D_IN_LINK_CONF_CH1_REG	Configures the receive descriptor operations of RX channel 1	0x061C	varies
DMA2D_IN_LINK_ADDR_CH1_REG	Configures the receive descriptor address of RX channel 1	0x0620	R/W
DMA2D_IN_ARB_CH1_REG	Configures the arbitration of RX channel 1	0x0640	R/W

Name	Description	Address	Access
DMA2D_IN_ETM_CONF_CH1_REG	Configures the ETM of RX channel 1	0x0648	R/W
DMA2D_IN_CONFO_CH2_REG	Configures the RX channel 2	0x0700	R/W
DMA2D_IN_POP_CH2_REG	Configures the FIFO of RX channel 2	0x0718	varies
DMA2D_IN_LINK_CONF_CH2_REG	Configures the receive descriptor operations of RX channel 2	0x071C	varies
DMA2D_IN_LINK_ADDR_CH2_REG	Configures the receive descriptor address of RX channel 2	0x0720	R/W
DMA2D_IN_ARB_CH2_REG	Configures the arbitration of RX channel 2	0x0740	R/W
DMA2D_IN_ETM_CONF_CH2_REG	Configures the ETM of RX channel 2	0x0748	R/W
DMA2D_RST_CONF_REG	Configures the reset of AXI	0x0A04	R/W
DMA2D_INTR_MEM_START_ADDR_REG	The start address of accessible internal memory address space	0x0A08	R/W
DMA2D_INTR_MEM_END_ADDR_REG	The end address of accessible internal memory address space	0x0A0C	R/W
DMA2D_EXTR_MEM_START_ADDR_REG	The start address of accessible external memory address space	0x0A10	R/W
DMA2D_EXTR_MEM_END_ADDR_REG	The end address of accessible external memory address space	0x0A14	R/W
DMA2D_OUT_ARB_CONFIG_REG	Configures the arbitration in TX direction	0x0A18	R/W
DMA2D_IN_ARB_CONFIG_REG	Configures the arbitration in RX direction	0x0A1C	R/W
Interrupt Registers			
DMA2D_OUT_INT_RAW_CHO_REG	Raw interrupt status of TX channel 0	0x0004	R/WTC/SS
DMA2D_OUT_INT_ENA_CHO_REG	Interrupt enable bits of TX channel 0	0x0008	R/W
DMA2D_OUT_INT_ST_CHO_REG	Masked interrupt status of TX channel 0	0x000C	RO
DMA2D_OUT_INT_CLR_CHO_REG	Interrupt clear bits of TX channel 0	0x0010	WT
DMA2D_OUT_INT_RAW_CH1_REG	Raw interrupt status of TX channel 1	0x0104	R/WTC/SS
DMA2D_OUT_INT_ENA_CH1_REG	Interrupt enable bits of TX channel 1	0x0108	R/W
DMA2D_OUT_INT_ST_CH1_REG	Masked interrupt status of TX channel 1	0x010C	RO
DMA2D_OUT_INT_CLR_CH1_REG	Interrupt clear bits of TX channel 1	0x0110	WT
DMA2D_OUT_INT_RAW_CH2_REG	Raw interrupt status of TX channel 2	0x0204	R/WTC/SS
DMA2D_OUT_INT_ENA_CH2_REG	Interrupt enable bits of TX channel 2	0x0208	R/W
DMA2D_OUT_INT_ST_CH2_REG	Masked interrupt status of TX channel 2	0x020C	RO
DMA2D_OUT_INT_CLR_CH2_REG	Interrupt clear bits of TX channel 2	0x0210	WT
DMA2D_OUT_INT_RAW_CH3_REG	Raw interrupt status of TX channel 3	0x0304	R/WTC/SS
DMA2D_OUT_INT_ENA_CH3_REG	Interrupt enable bits of TX channel 3	0x0308	R/W

Name	Description	Address	Access
DMA2D_OUT_INT_ST_CH3_REG	Masked interrupt status of TX channel 3	0x030C	RO
DMA2D_OUT_INT_CLR_CH3_REG	Interrupt clear bits of TX channel 3	0x0310	WT
DMA2D_IN_INT_RAW_CHO_REG	Raw interrupt status of RX channel 0	0x0504	R/WTC/SS
DMA2D_IN_INT_ENA_CHO_REG	Interrupt enable bits of RX channel 0	0x0508	R/W
DMA2D_IN_INT_ST_CHO_REG	Masked interrupt status of RX channel 0	0x050C	RO
DMA2D_IN_INT_CLR_CHO_REG	Interrupt clear bits of RX channel 0	0x0510	WT
DMA2D_IN_INT_RAW_CH1_REG	Raw interrupt status of RX channel 1	0x0604	R/WTC/SS
DMA2D_IN_INT_ENA_CH1_REG	Interrupt enable bits of RX channel 1	0x0608	R/W
DMA2D_IN_INT_ST_CH1_REG	Masked interrupt status of RX channel 1	0x060C	RO
DMA2D_IN_INT_CLR_CH1_REG	Interrupt clear bits of RX channel 1	0x0610	WT
DMA2D_IN_INT_RAW_CH2_REG	Raw interrupt status of RX channel 2	0x0704	R/WTC/SS
DMA2D_IN_INT_ENA_CH2_REG	Interrupt enable bits of RX channel 2	0x0708	R/W
DMA2D_IN_INT_ST_CH2_REG	Masked interrupt status of RX channel 2	0x070C	RO
DMA2D_IN_INT_CLR_CH2_REG	Interrupt clear bits of RX channel 2	0x0710	WT
Status Registers			
DMA2D_OUTFIFO_STATUS_CHO_REG	Represents the status of the FIFO of TX channel 0	0x0014	RO
DMA2D_OUT_STATE_CHO_REG	Represents the working status of the transmit descriptor of TX channel 0	0x0024	RO
DMA2D_OUT_EOF_DES_ADDR_CHO_REG	Represents the transmit descriptor address when EOF occurs on TX channel 0	0x0028	RO
DMA2D_OUT_DSCR_CHO_REG	Represents the address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 0	0x002C	RO
DMA2D_OUT_DSCR_BFO_CHO_REG	Represents the address of the current pre-read transmit descriptor on TX channel 0	0x0030	RO
DMA2D_OUT_DSCR_BF1_CHO_REG	Represents the address of the previous pre-read transmit descriptor on TX channel 0	0x0034	RO
DMA2D_OUTFIFO_STATUS_CH1_REG	Represents the status of the FIFO of TX channel 1	0x0114	RO
DMA2D_OUT_STATE_CH1_REG	Represents the working status of the transmit descriptor of TX channel 1	0x0124	RO
DMA2D_OUT_EOF_DES_ADDR_CH1_REG	Represents the transmit descriptor address when EOF occurs on TX channel 1	0x0128	RO

Name	Description	Address	Access
DMA2D_OUT_DSCR_CH1_REG	Represents the address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 1	0x012C	RO
DMA2D_OUT_DSCR_BFO_CH1_REG	Represents the address of the current pre-read transmit descriptor on TX channel 1	0x0130	RO
DMA2D_OUT_DSCR_BF1_CH1_REG	Represents the address of the previous pre-read transmit descriptor on TX channel 1	0x0134	RO
DMA2D_OUTFIFO_STATUS_CH2_REG	Represents the status of the FIFO of TX channel 2	0x0214	RO
DMA2D_OUT_STATE_CH2_REG	Represents the working status of the transmit descriptor of TX channel 2	0x0224	RO
DMA2D_OUT_EOF_DES_ADDR_CH2_REG	Represents the transmit descriptor address when EOF occurs on TX channel 2	0x0228	RO
DMA2D_OUT_DSCR_CH2_REG	Represents the address of the next transmit descriptor pointed by the current pre-read transmit descriptor on TX channel 2	0x022C	RO
DMA2D_OUT_DSCR_BFO_CH2_REG	Represents the address of the current pre-read transmit descriptor on TX channel 2	0x0230	RO
DMA2D_OUT_DSCR_BF1_CH2_REG	Represents the address of the previous pre-read transmit descriptor on TX channel 2	0x0234	RO
DMA2D_INFIFO_STATUS_CHO_REG	Represents the status of the FIFO of RX channel 0	0x0514	RO
DMA2D_IN_STATE_CHO_REG	Represents the working status of the receive descriptor of RX channel 0	0x0524	RO
DMA2D_IN_SUC_EOF_DES_ADDR_CHO_REG	Represents the receive descriptor address when EOF occurs on RX channel 0	0x0528	RO
DMA2D_IN_ERR_EOF_DES_ADDR_CHO_REG	Represents the receive descriptor address when errors occur on RX channel 0	0x052C	RO
DMA2D_IN_DSCR_CHO_REG	Represents the address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 0	0x0530	RO
DMA2D_IN_DSCR_BFO_CHO_REG	Represents address of the current pre-read receive descriptor on RX channel 0	0x0534	RO
DMA2D_IN_DSCR_BF1_CHO_REG	Represents the address of the previous pre-read receive descriptor on RX channel 0	0x0538	RO
DMA2D_INFIFO_STATUS_CH1_REG	Represents the status of the FIFO of RX channel 1	0x0614	RO

Name	Description	Address	Access
DMA2D_IN_STATE_CH1_REG	Represents the working status of the receive descriptor of RX channel 1	0x0624	RO
DMA2D_IN_SUC_EOF_DES_ADDR_CH1_REG	Represents the receive descriptor address when EOF occurs on RX channel 1	0x0628	RO
DMA2D_IN_ERR_EOF_DES_ADDR_CH1_REG	Represents the receive descriptor address when errors occur on RX channel 1	0x062C	RO
DMA2D_IN_DSCR_CH1_REG	Represents the address of the next receive descriptor pointed by the current pre-read receive descriptor on RX channel 1	0x0630	RO
DMA2D_IN_DSCR_BFO_CH1_REG	Represents address of the current pre-read receive descriptor on RX channel 1	0x0634	RO
DMA2D_IN_DSCR_BF1_CH1_REG	Represents the address of the previous pre-read receive descriptor on RX channel 1	0x0638	RO
DMA2D_AXI_ERR_REG	Represents the status of the AXI bus	0x0A00	RO
DMA2D_DATE_REG	Version register	0x0A2C	R/W
Peripheral Select Registers			
DMA2D_OUT_PERI_SEL_CHO_REG	Configures the peripheral connected to TX channel 0	0x0038	R/W
DMA2D_OUT_PERI_SEL_CH1_REG	Configures the peripheral connected to TX channel 1	0x0138	R/W
DMA2D_OUT_PERI_SEL_CH2_REG	Configures the peripheral connected to TX channel 2	0x0238	R/W
DMA2D_OUT_PERI_SEL_CH3_REG	Configures the peripheral connected to TX channel 3	0x0338	R/W
DMA2D_IN_PERI_SEL_CHO_REG	Configures the peripheral connected to RX channel 0	0x053C	R/W
DMA2D_IN_PERI_SEL_CH1_REG	Configures the peripheral connected to RX channel 1	0x063C	R/W
DMA2D_IN_PERI_SEL_CH2_REG	Configures the peripheral connected to RX channel 2	0x073C	R/W

6.9 Registers

The addresses in this section are relative to 2D-DMA base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 6.1. DMA2D_OUT_CONFO_CH n _REG (n : 0-3) (0x0000+0x100* n)

(reserved)						DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CH ⁿ						(reserved)						DMA2D_OUT_CMD_DISABLE_CH ⁿ						(reserved)						DMA2D_OUT_REORDER_EN_CHO						(reserved)						DMA2D_OUT_DSCR_PORT_EN_CH ⁿ						DMA2D_OUT_MACRO_BLOCK_SIZE_CH ⁿ						(reserved)						DMA2D_OUT_MEM_BURST_LENGTH_CH ⁿ						(reserved)						DMA2D_OUT_CHECK_OWNER_CH ⁿ						DMA2D_OUTDSCR_ECC_AES_EN_CH ⁿ						DMA2D_OUT_EOF_MODE_CH ⁿ						DMA2D_OUT_AUTO_WBACK_CH ⁿ					
31					27	26	25	24	23					17	16	15					12	11	10	9	8					6	5	4	3	2	1	0	Reset																																																										
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x0	0x0				0	0	0	0	0	0	1	0																																																														

DMA2D_OUT_AUTO_WRBACK_CH_n Configures whether to enable automatic outlink write-back when all the data in TX FIFO has been transmitted.

0: Disable

1: Enable

(R/W)

DMA2D_OUT_EOF_MODE_CH n Configures when to generate EOF flag.

0: EOF flag for TX channel *n* is generated when data need to read has been pushed into FIFO of 2DDMA

1: EOF flag for TX channel *n* is generated when data need to read has been popped from FIFO of 2DDMA

(R/W)

DMA2D_OUTDSCR_BURST_EN_CH n Configures whether to enable INCR burst transfer for TX channel n reading descriptors.

0: Disable

1: Enable

(R/W)

DMA2D_OUT_ECC_AES_EN_CH*n* Configures to enable the access to external memory space for ECC and AES (encrypted) via TX channel *n*.

0: Disable

1: Enable

In this case, the starting address of the space and the corresponding data should be 16-byte aligned.

(R/W)

Continued on the next page...

Register 6.1. DMA2D_OUT_CONFO_CH n _REG (n : 0-3) (0x0000+0x100* n)

Continued from the previous page...

DMA2D_OUT_CHECK_OWNER_CH n Configures whether to enable the owner bit check for TX channel n .

0: Disable

1: Enable

(R/W)

DMA2D_OUT_MEM_BURST_LENGTH_CH n Configures the burst length for TX channel n .

0: 8 bytes

1: 16 bytes

2: 32 bytes

3: 64 bytes

4: 128 bytes

Others: Invalid

(R/W)

DMA2D_OUT_MACRO_BLOCK_SIZE_CH n Configures macroblock size for TX channel n . Used only in 2D-MOD1 mode.

0: 8 * 8 pixels

1: 8 * 16 pixels

2: 16 * 16 pixels

3: No macroblock

(R/W)

DMA2D_OUT_DSCR_PORT_EN_CH n Configures whether to enable the DSCR-PORT mode for TX channel n .

0: Disable

1: Enable

(R/W)

DMA2D_OUT_REORDER_EN_CH0 Configures whether to enable macroblock reordering for TX channel 0.

0: Disable

1: Enable

(R/W)

DMA2D_OUT_RST_CH n Configures whether to reset TX channel n .

0: Reset release

1: Reset

(R/W)

Continued on the next page...

Register 6.1. DMA2D_OUT_CONFO_CH n _REG (n : 0-3) (0x0000+0x100* n)

Continued from the previous page...

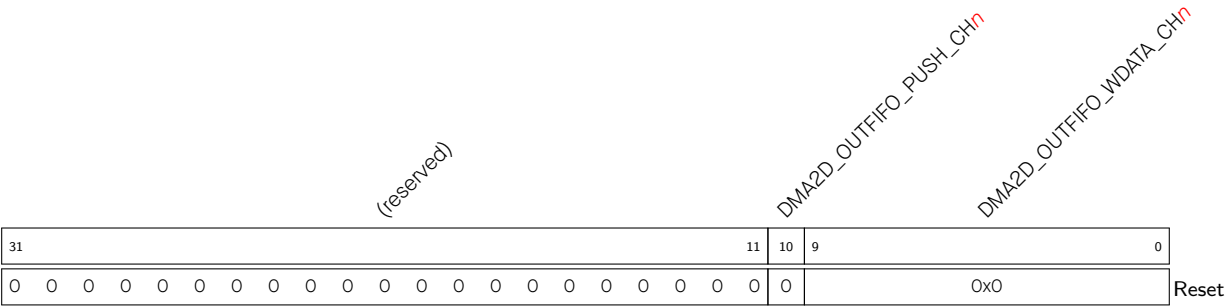
DMA2D_OUT_CMD_DISABLE_CH n Configures reset command for TX channel n .

- 0: Set 0 after reset release
 - 1: Pause transfers before reset
- (R/W)

DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CH n Configures whether to enable weight optimization for TX channel n .

- 0: Enable
 - 1: Disable
- (R/W)

Register 6.2. DMA2D_OUT_PUSH_CH n _REG (n : 0-3) (0x0018+0x100* n)

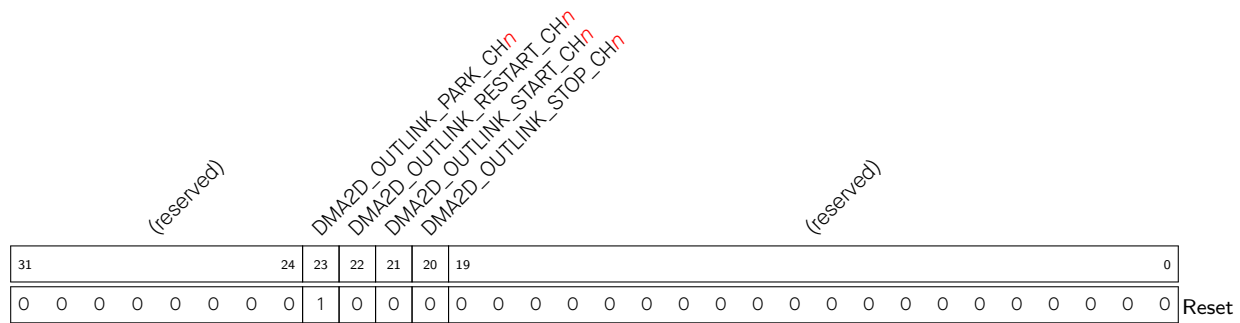


DMA2D_OUTFIFO_WDATA_CH n Configures the data to be pushed into the 2D-DMA TX FIFO. (R/W)

DMA2D_OUTFIFO_PUSH_CH n Configures whether to push data into the 2D-DMA TX FIFO.

- 0: Not push 1: Push
- (R/W/SC)

Register 6.3. DMA2D_OUT_LINK_CONF_CH*n*_REG (*n*: 0-3) (0x001C+0x100**n*)



DMA2D_OUTLINK_STOP_CH*n* Configures whether to stop TX channel *n* from transmitting data.

0: Invalid. No effect

1: Stop

(R/W/SC)

DMA2D_OUTLINK_START_CH n Configures whether to enable TX channel n for data transfer.

0: Disable

1: Enable

(R/W/SC)

DMA2D_OUTLINK_RESTART_CH*n* Configures whether to restart TX channel *n* for AXI DMA transfer.

0: Invalid. No effect

1: Restart

(R/W/SC)

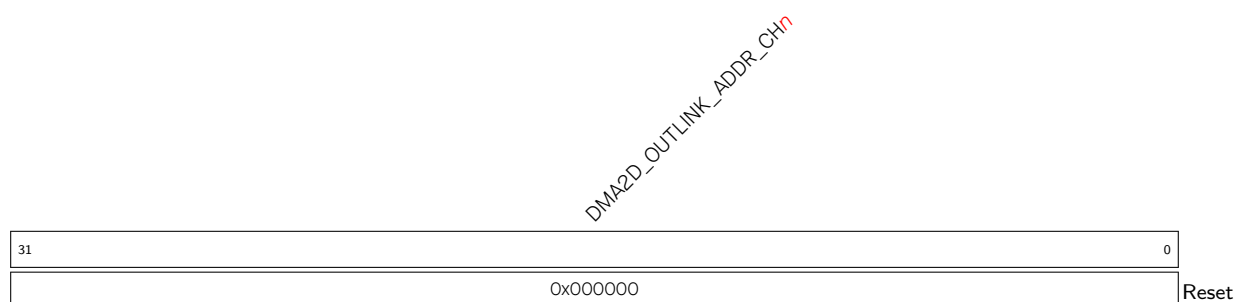
DMA2D_OUTLINK_PARK_CH_n Represents the status of the transmit descriptor's FSM.

0: Running

1: Idle

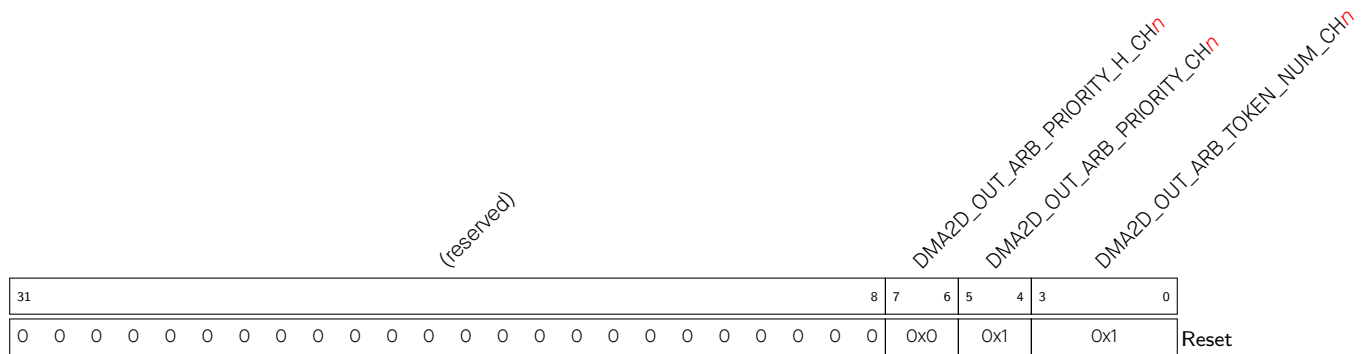
(RO)

Register 6.4. DMA2D_OUT_LINK_ADDR_CH n _REG (n : 0-3) (0x0020+0x100* n)



DMA2D_OUTLINK_ADDR_CH_n Represents the first transmit descriptor's address. (R/W)

Register 6.5. DMA2D_OUT_ARB_CH n _REG (n : 0-3) (0x003C+0x100* n)



DMA2D_OUT_ARB_TOKEN_NUM_CH n Configures the weight (i.e the number of tokens) of TX channel n .

Value range: 0 ~ 15. (R/W)

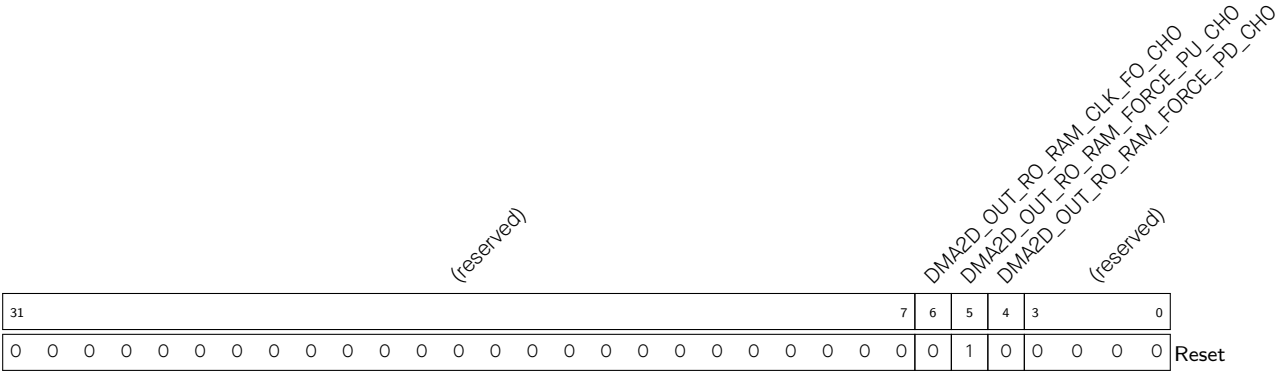
DMA2D_OUT_ARB_PRIORITY_CH n Configures the lower two bits of TX channel n priority.

Together with DMA2D_OUT_ARB_PRIORITY_H_CH n , this field forms a 4-bit priority value for TX channel n , ranging from 0 (lowest) to 15 (highest). (R/W)

DMA2D_OUT_ARB_PRIORITY_H_CH n Configures the higher two bits of TX channel n priority.

Together with DMA2D_OUT_ARB_PRIORITY_CH n , this field forms a 4-bit priority value for TX channel n , ranging from 0 (lowest) to 15 (highest). (R/W)

Register 6.6. DMA2D_OUT_RO_PD_CONF_CHO_REG (0x0044)



DMA2D_OUT_RO_RAM_FORCE_PD_CHO Configures whether to force power down the macroblock reordering RAM for TX channel 0.

0: Not force power down

1: Force power down

(R/W)

DMA2D_OUT_RO_RAM_FORCE_PU_CHO Configures whether to force power up the macroblock reordering RAM for TX channel 0.

0: Not force power up

1: Force power up

(R/W)

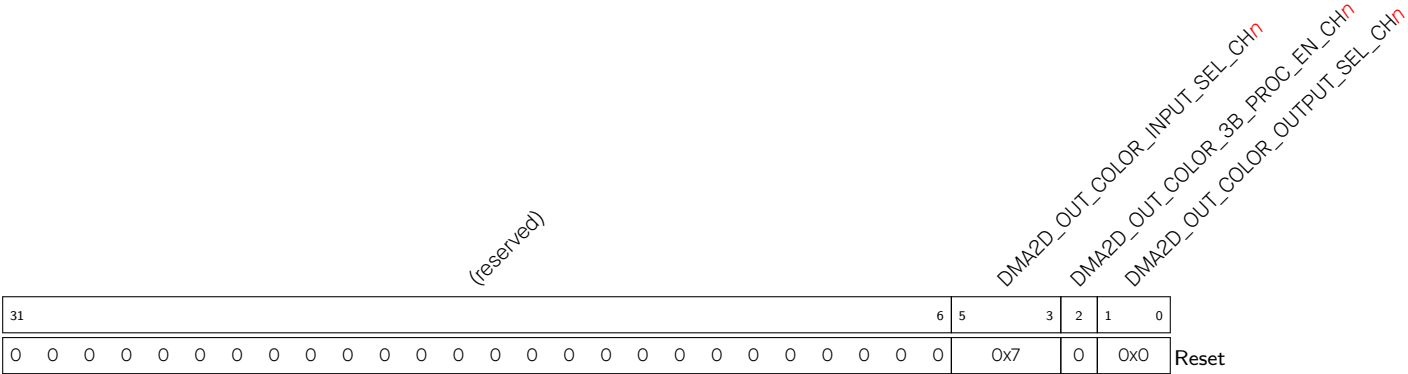
DMA2D_OUT_RO_RAM_CLK_FO_CHO Configures the clock gating when TX channel 0 accessing the 2D-DMA RAM.

0: Use the clock gate

1: Force the clock on and bypass the clock gate

(R/W)

Register 6.7. DMA2D_OUT_COLOR_CONVERT_CH n _REG (n : 0-3) (0x0048+0x100* n)



DMA2D_OUT_COLOR_OUTPUT_SEL_CH n Configures the third stage output of color space conversion for TX channel n .

- 0: RGB888 to RGB565
- 1: YUV444 to YUV422
- 2: Output directly
- Others: Invalid

(R/W)

DMA2D_OUT_COLOR_3B_PROC_EN_CH n Configures whether to enable the second stage of color space conversion for TX channel n .

- 0: Disable
- 1: Enable

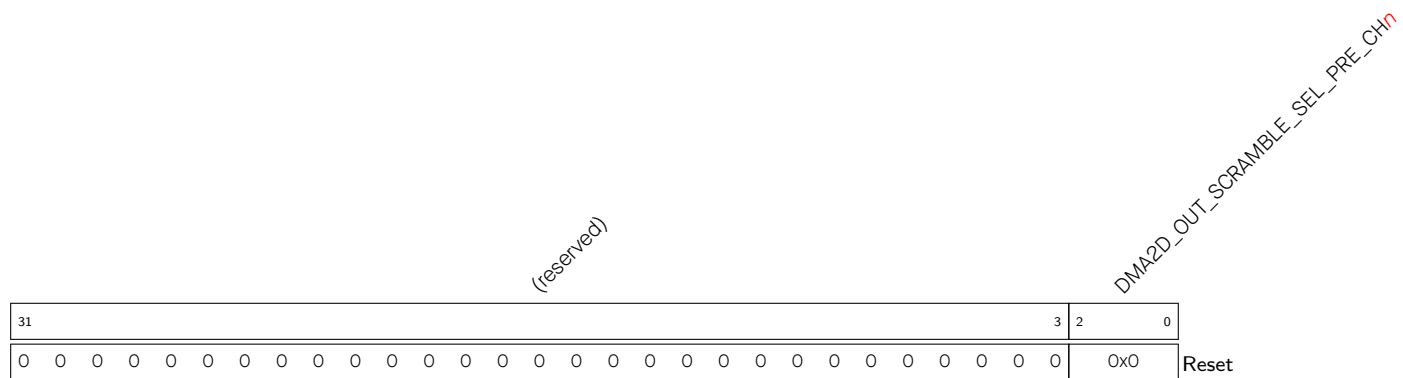
(R/W)

DMA2D_OUT_COLOR_INPUT_SEL_CH n Configures the first stage output of color space conversion for TX channel n .

- 0: RGB565 to RGB888
- 1: YUV422 to YUV444
- 2: Other 2 bytes/pixel color format
- 3: Other 3 bytes/pixel color format
- 7: Disable color space conversion
- Others: Invalid

(R/W)

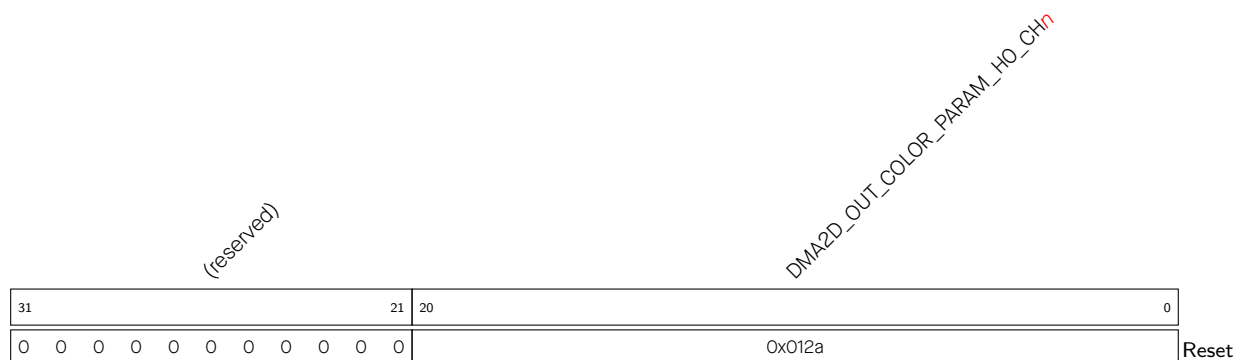
Register 6.8. DMA2D_OUT_SCRAMBLE_CH n _REG (n : 0-3) (0x004C+0x100* n)



DMA2D_OUT_SCRAMBLE_SEL_PRE_CH n Configures the byte order scrambling before the color space conversion for TX channel n .

- 0: BYTE2-1-0
1: BYTE2-0-1
2: BYTE1-0-2
3: BYTE1-2-0
4: BYTE0-2-1
5: BYTE0-1-2
Others: Invalid
(R/W)

Register 6.9. DMA2D_OUT_COLOR_PARAM0_CH n _REG (n : 0-3) (0x0050+0x100* n)



DMA2D_OUT_COLOR_PARAM_HO_CH n Configures the coefficient A and B for the most significant byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.10. DMA2D_OUT_COLOR_PARAM1_CH*n*_REG (*n*: 0-3) (0x0054+0x100**n*)

Diagram illustrating the structure of the `DMA2D_OUT_COLOR_PARAM_H1_CHn` register. The register is 32 bits wide, divided into a 4-bit reserved field (bits 31-28) and a 28-bit data field (bits 27-0). The data field contains the value `0xc86d999`.

DMA2D_OUT_COLOR_PARAM_H1_CH n Configures the coefficient C and D for the most significant byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.11. DMA2D_OUT_COLOR_PARAM2_CH n _REG (n : 0-3) (0x0058+0x100* n)

Diagram illustrating the structure of the `DMA2D_OUT_COLOR_PARAM_MO_CHn` register. The register is 32 bits wide, divided into a 21-bit reserved field and an 11-bit field for `DMA2D_OUT_COLOR_PARAM_MO_CHn`. The value `0x1e712a` is shown in the lower 11 bits.

31	21	20	0
(reserved)		<code>DMA2D_OUT_COLOR_PARAM_MO_CHn</code>	
0 0 0 0 0 0 0 0 0 0 0 0		<code>0x1e712a</code>	

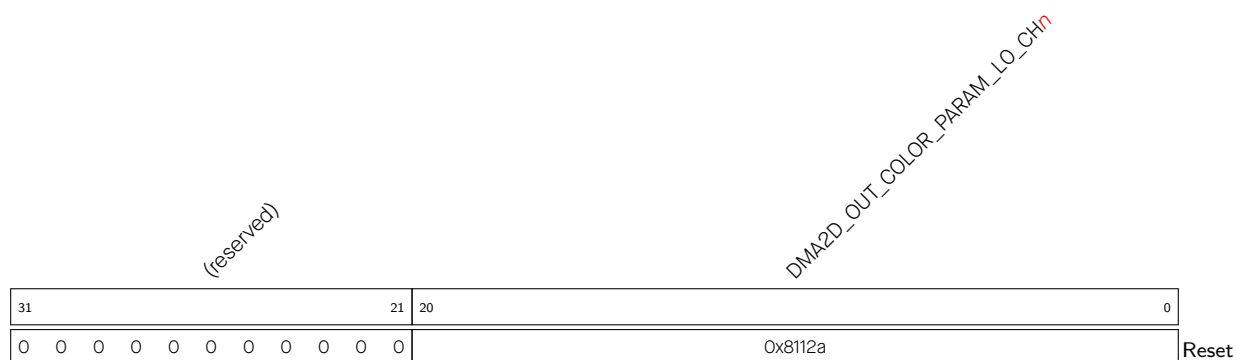
DMA2D_OUT_COLOR_PARAM_MO_CH n Configures the coefficient A and B for the middle byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.12. DMA2D_OUT_COLOR_PARAM3_CH n _REG (n : 0-3) (0x005C+0x100* n)

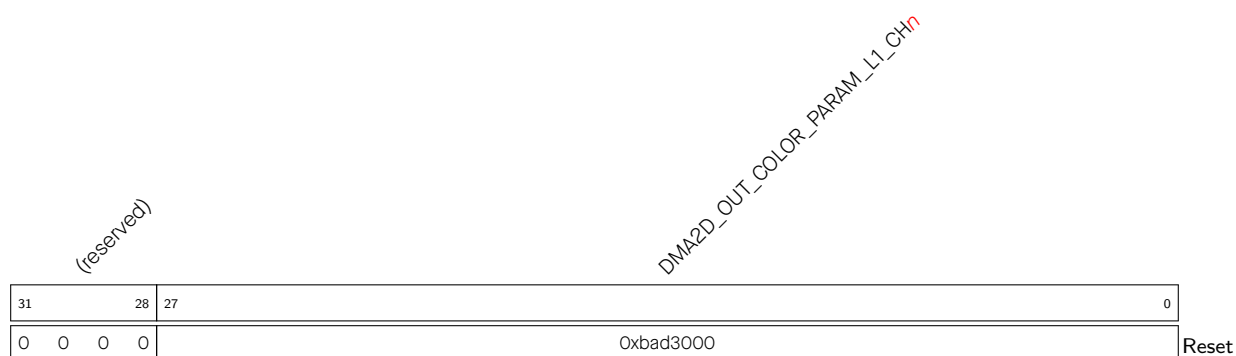
31	28	27	0
0	0	0	0
		0x21e4f30	

Reset

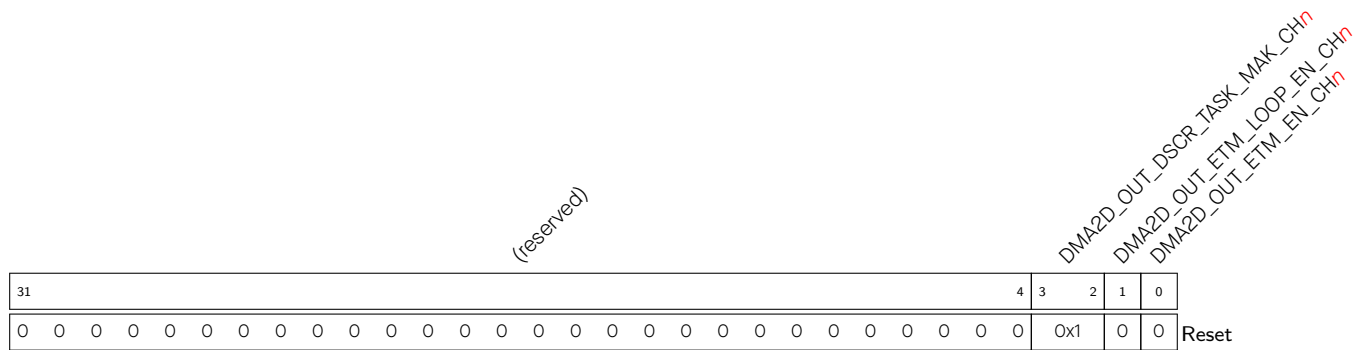
DMA2D_OUT_COLOR_PARAM_M1_CH n Configures the coefficient C and D for the middle byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.13. DMA2D_OUT_COLOR_PARAM4_CH n _REG (n : 0-3) (0x0060+0x100* n)

DMA2D_OUT_COLOR_PARAM_LO_CH n Configures the coefficient A and B for the least significant byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.14. DMA2D_OUT_COLOR_PARAM5_CH n _REG (n : 0-3) (0x0064+0x100* n)

DMA2D_OUT_COLOR_PARAM_L1_CH n Configures the coefficient C and D for the least significant byte of the input of color space conversion's second stage for TX channel n . (R/W)

Register 6.15. DMA2D_OUT_ETM_CONF_CH n _REG (n : 0-3) (0x0068+0x100* n)

DMA2D_OUT_ETM_EN_CH n Configures whether to enable the ETM function for TX channel n .

0: Disable

1: Enable

(R/W)

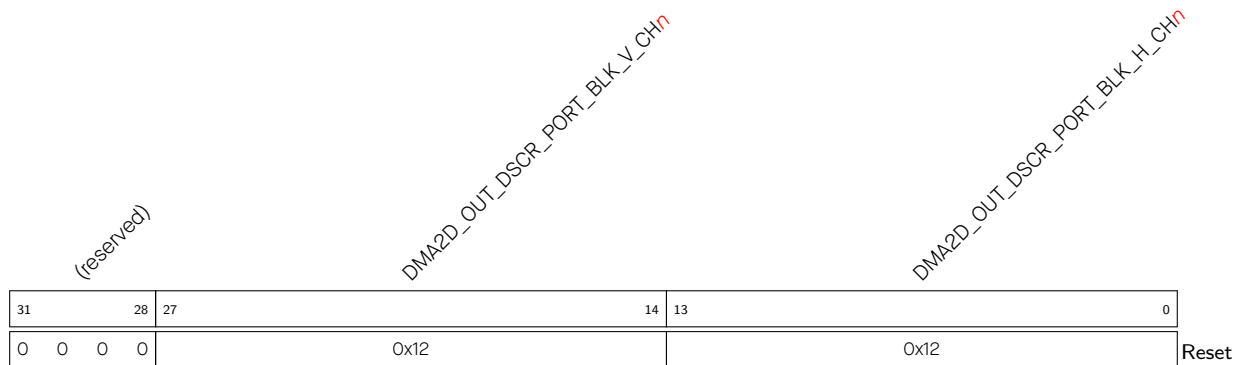
DMA2D_OUT_ETM_LOOP_EN_CH n Configures whether to use the ETM task to indicate the processing of the next descriptor for TX channel n .

0: Not use ETM task

1: Use ETM task

(R/W)

DMA2D_OUT_DSCR_TASK_MAK_CH n Configures the maximum number of tasks that can be cached for TX channel n . (R/W)

Register 6.16. DMA2D_OUT_DSCR_PORT_BLK_CH n _REG (n : 0-3) (0x006C+0x100* n)

DMA2D_OUT_DSCR_PORT_BLK_H_CH n Configures the horizontal width of the basic unit for TX channel n in DSCR-PORT mode.

Measurement unit: pixels. (R/W)

DMA2D_OUT_DSCR_PORT_BLK_V_CH n Configures the vertical height of the basic unit for TX channel n in DSCR-PORT mode.

Measurement unit: pixels. (R/W)

Register 6.17. DMA2D_IN_CONFO_CH n _REG (n : 0-2) (0x0500+0x100* n)

(reserved)						DMA2D_IN_ARB_WEIGHT_OPT_DIS_CH ⁿ				(reserved)								DMA2D_IN_REORDER_EN_CHO				(reserved)						DMA2D_IN_DSCR_PORT_EN_CH ⁿ				DMA2D_IN_MACRO_BLOCK_SIZE_CH ⁿ				reserved				DMA2D_IN_CHECK_OWNER_CH ⁿ				DMA2D_IN_ECC_AES_EN_CH ⁿ				DMA2D_INDSCR_BURST_EN_CH ⁿ				(reserved)				DMA2D_IN_MEM_TRANS_EN_CH ⁿ			
31					27		26	25	24	23					17		16	15	12			11	10	9	8	6		5	4	3	2	1	0																										
0					0		0	0	0	0					0		0	0	0	0			0	0x0		0x0		0	0	0	0	0	0	Reset																									

DMA2D_IN_MEM_TRANS_EN_CH n Configures whether to enable memory-to-memory data transfer for TX and RX channel n .

0: Disable

1: Enable

(R/W)

DMA2D_INDSCR_BURST_EN_CH n Configures whether to enable INCR burst transfer for RX channel n to read descriptors when accessing internal memory.

0: Disable

1: Enable

(R/W)

DMA2D_IN_ECC_AES_EN_CH*n* Configures whether to enable the access to external memory space for ECC and AES (encrypted) via RX channel *n*.

0: Disable

1: Enable. In this case, the starting address of the space and the corresponding data should be 16-byte aligned.

(R/W)

DMA2D_IN_CHECK_OWNER_CH*n* Configures whether to enable the owner bit check for RX channel *n*.

0: Disable

1: Enable

(R/W)

DMA2D_IN_MEM_BURST_LENGTH_CH n Configures the burst length for RX channel n .

0: 8 bytes

1: 16 bytes

2: 32 bytes

3: 64 bytes

4: 128 bytes

Others: Invalid

(R/W)

Continued on the next page...

Register 6.17. DMA2D_IN_CONFO_CH n _REG (n : 0-2) (0x0500+0x100* n)

Continued from the previous page...

DMA2D_IN_MACRO_BLOCK_SIZE_CH n Configures macroblock size for RX channel n . Used only in 2D-MOD1 mode.

0: 8 * 8 pixels

1: 8 * 16 pixels

2: 16 * 16 pixels

3: No macroblock

(R/W)

DMA2D_IN_DSCR_PORT_EN_CH n Configures whether to enable the DSCR-PORT mode for RX channel n .

0: Disable

1: Enable

(R/W)

DMA2D_IN_REORDER_EN_CHO Configures whether to enable macroblock reordering for RX channel 0.

0: Disable

1: Enable

(R/W)

DMA2D_IN_RST_CH n Configures whether to reset RX channel n .

0: Invalid

1: Reset

(R/W)

DMA2D_IN_CMD_DISABLE_CH n Configures reset command for RX channel n .

0: Set 0 after reset release

1: Pause transfers before reset

(R/W)

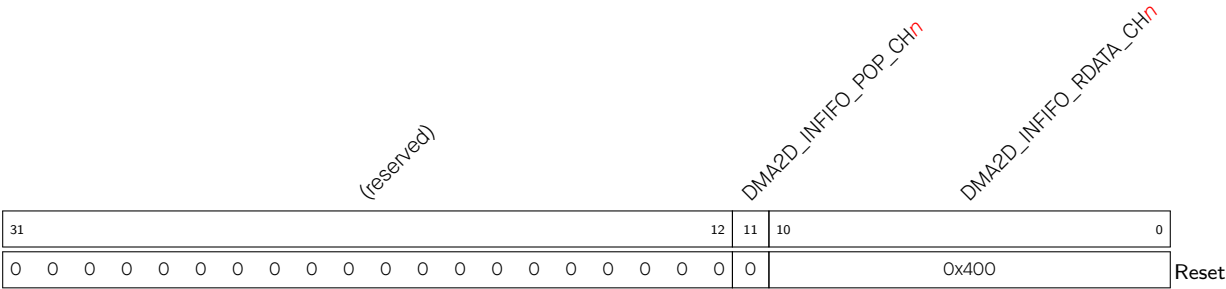
DMA2D_IN_ARB_WEIGHT_OPT_DIS_CH n Configures whether to enable weight optimization for RX channel n .

0: Enable

1: Disable

(R/W)

Register 6.18. DMA2D_IN_POP_CHn_REG (n: 0-2) (0x0518+0x100*n)



DMA2D_INFIFO_RDATA_CHn Represents the data popped from 2D-DMA RX FIFO. (RO)

DMA2D_INFIFO_POP_CHn Configures whether to pop data from 2D-DMA RX FIFO.

- 0: Not pop
 - 1: Pop
- (R/W/SC)

Register 6.19. DMA2D_IN_LINK_CONF_CH n _REG (n : 0-2) (0x051C+0x100* n)

(reserved)								DMA2D_INLINK_PARK_CH _n DMA2D_INLINK_RESTART_CH _n DMA2D_INLINK_STOP_CH _n DMA2D_INLINK_AUTO_RET_CH _n																			(reserved)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
31					25	24	23	22	21	20	19																						0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

DMA2D_INLINK_AUTO_RET_CH n Configure the value of the owner bit written back to the receive descriptor.

0: Write back 0

1: Write back 1

(R/W)

DMA2D_INLINK_STOP_CH n Configures whether to stop RX channel n from receiving data.

0: Invalid. No effect

1: Stop

(R/W/SC)

DMA2D_INLINK_START_CH n Configures whether to enable RX channel n for data transfer.

0: Disable

1: Enable

(R/W/SC)

DMA2D_INLINK_RESTART_CH n Configures whether to restart RX channel n for 2D-DMA transfer.

0: Invalid. No effect

1: Restart

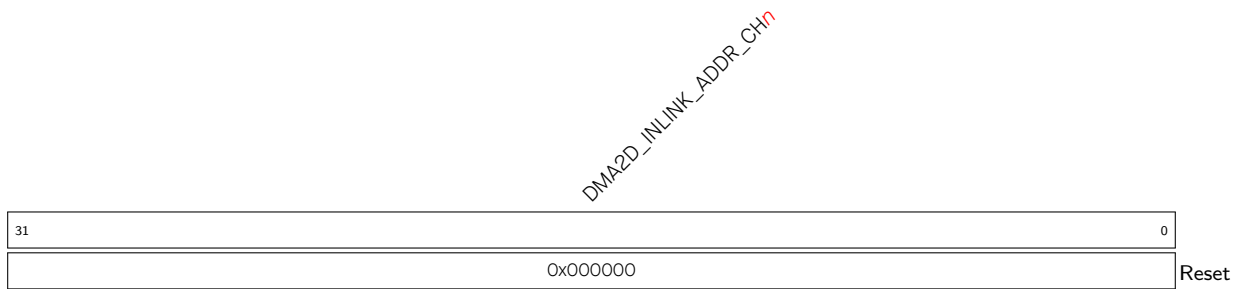
(R/W/SC)

DMA2D_INLINK_PARK_CH n Represents the status of the receive descriptor's FSM.

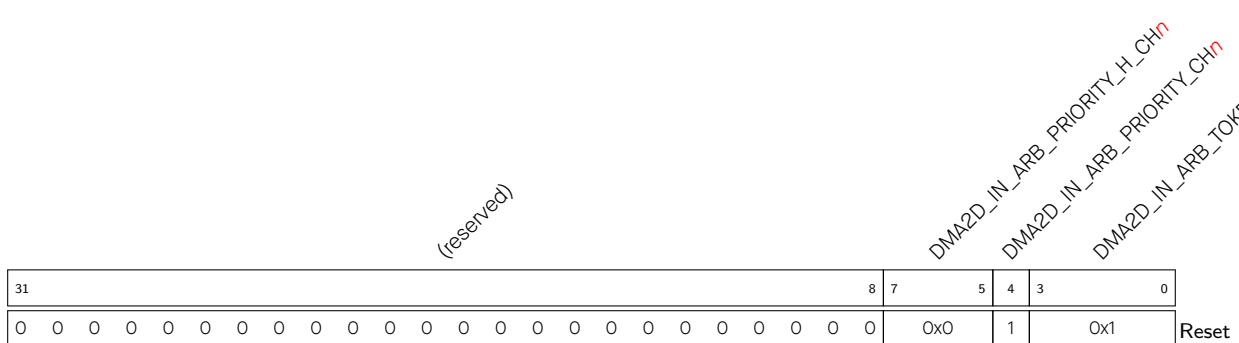
0: Running

1: Idle

(RO)

Register 6.20. DMA2D_IN_LINK_ADDR_CH n _REG (n : 0-2) (0x0520+0x100* n)

DMA2D_INLINK_ADDR_CH n Represents the first receive descriptor's address. (R/W)

Register 6.21. DMA2D_IN_ARB_CH n _REG (n : 0-2) (0x0540+0x100* n)

DMA2D_IN_ARB_TOKEN_NUM_CH n Configures the weight (i.e the number of tokens) of RX channel n .

Value range: 0 ~ 15. (R/W)

DMA2D_IN_ARB_PRIORITY_CH n Configures the lower two bits of RX channel n priority.

Together with DMA2D_IN_ARB_PRIORITY_H_CH n , this field forms a 4-bit priority value for TX channel n , ranging from 0 (lowest) to 15 (highest). (R/W)

DMA2D_IN_ARB_PRIORITY_H_CH n Configures the higher two bits of TX channel n priority.

Together with DMA2D_IN_ARB_PRIORITY_CH n , this field forms a 4-bit priority value for TX channel n , ranging from 0 (lowest) to 15 (highest). (R/W)

Register 6.22. DMA2D_IN_RO_PD_CONF_CHO_REG (0x0548)

(reserved)																												7	6	5	4	3	0														
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	Reset													
																																DMA2D_IN_RO_RAM_CLK_FO_CHO				DMA2D_IN_RO_RAM_FORCE_PU_CHO				DMA2D_IN_RO_RAM_FORCE_PD_CHO							

DMA2D_IN_RO_RAM_FORCE_PD_CHO	Configures whether to force power down the macroblock reordering RAM for RX channel 0.
-------------------------------------	--

0: Not force power down

1: Force power down

(R/W)

DMA2D_IN_RO_RAM_FORCE_PU_CHO	Configures whether to force power up the macroblock re-ordering RAM for RX channel 0.
-------------------------------------	---

0: Not force power up

1: Force power up

(R/W)

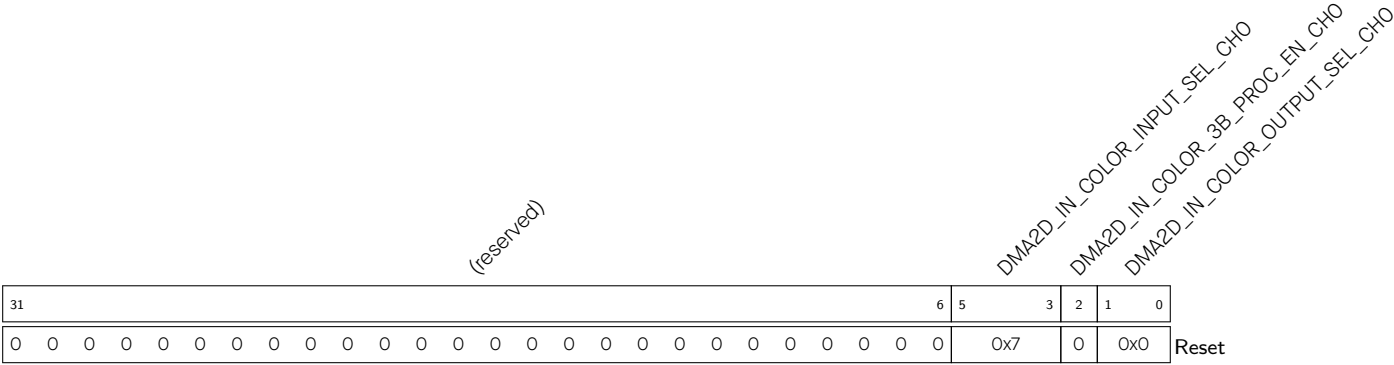
DMA2D_IN_RO_RAM_CLK_FO_CHO Configures the clock gating when RX channel 0 accessing the 2D-DMA RAM.

0: Use the clock gate

1: Force the clock on and bypass the clock gate

(R/W)

Register 6.23. DMA2D_IN_COLOR_CONVERT_CHO_REG (0x054C)



DMA2D_IN_COLOR_OUTPUT_SEL_CHO Configures the third stage output of color space conversion for RX channel 0.

0: RGB888 to RGB565

1: Output directly

(R/W)

DMA2D_IN_COLOR_3B_PROC_EN_CHO Configures whether to enable the second stage of color space conversion for RX channel 0.

0: Disable

1: Enable

(R/W)

DMA2D_IN_COLOR_INPUT_SEL_CHO Configures the first stage output of color space conversion for RX channel 0.

0: YUV422/420 to YUV444

1: YUV422

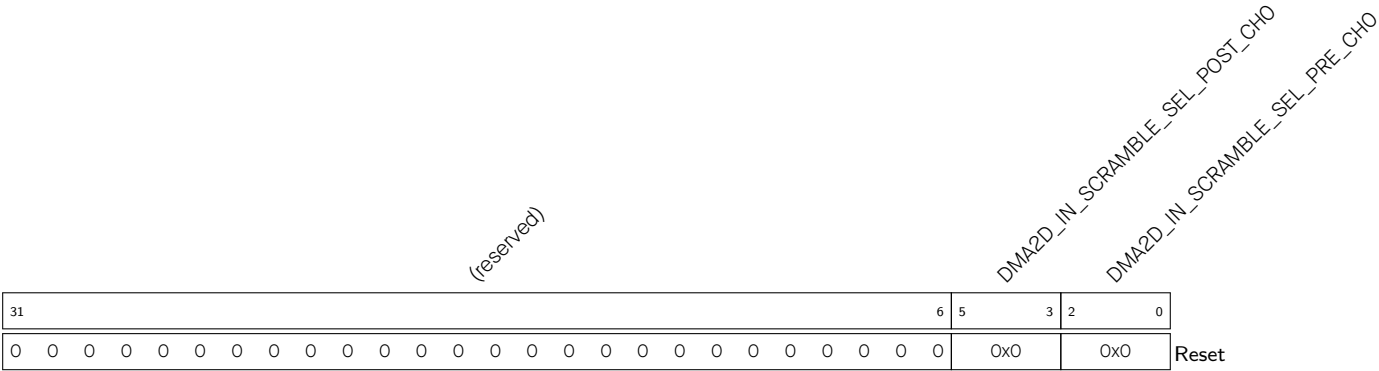
2: YUV444/420

7: Disable color space conversion

Others: Invalid

(R/W)

Register 6.24. DMA2D_IN_SCRAMBLE_CHO_REG (0x0550)



DMA2D_IN_SCRAMBLE_SEL_PRE_CHO Configures the byte order scrambling before the color space conversion for RX channel 0.

- 0: BYTE2-1-0
- 1: BYTE2-0-1
- 2: BYTE1-0-2
- 3: BYTE1-2-0
- 4: BYTE0-2-1
- 5: BYTE0-1-2
- Others: Invalid

(R/W)

DMA2D_IN_SCRAMBLE_SEL_POST_CHO Configures the byte order scrambling after the color space conversion for RX channel 0.

- 0: BYTE2-1-0
- 1: BYTE2-0-1
- 2: BYTE1-0-2
- 3: BYTE1-2-0
- 4: BYTE0-2-1
- 5: BYTE0-1-2
- Others: Invalid

(R/W)

Register 6.25. DMA2D_IN_COLOR_PARAMO_CHO_REG (0x0554)

(reserved)												DMA2D_IN_COLOR_PARAM_HO_CHO																																											
31												21												20																				0											
0 0 0 0 0 0 0 0 0 0 0 0												0x012a																				Reset																							

DMA2D_IN_COLOR_PARAM_HO_CHO Configures the coefficient A and B for the most significant byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.26. DMA2D_IN_COLOR_PARAM1_CHO_REG (0x0558)

(reserved)																DMA2D_IN_COLOR_PARAM_H1_CHO																			
31				28				27																				0							
0				0				0				0				0xc86d999																Reset			

DMA2D_IN_COLOR_PARAM_H1_CHO Configures the coefficient C and D for the most significant byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.27. DMA2D_IN_COLOR_PARAM2_CHO_REG (0x055C)

(reserved)												DMA2D_IN_COLOR_PARAM_MO_CHO																																											
31												21												20																				0											
0 0 0 0 0 0 0 0 0 0 0 0												0x1e712a																				Reset																							

DMA2D_IN_COLOR_PARAM_MO_CHO Configures the coefficient A and B for the middle byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.28. DMA2D_IN_COLOR_PARAM3_CHO_REG (0x0560)

(reserved)				DMA2D_IN_COLOR_PARAM_M1_CHO																												
31	28	27																													0	
0	0	0	0	0x21e4f30																												Reset

DMA2D_IN_COLOR_PARAM_M1_CHO Configures the coefficient C and D for the middle byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.29. DMA2D_IN_COLOR_PARAM4_CHO_REG (0x0564)

(reserved)												DMA2D_IN_COLOR_PARAM_LO_CHO																	
3121												200																	
000000000000												0x8112aReset																	

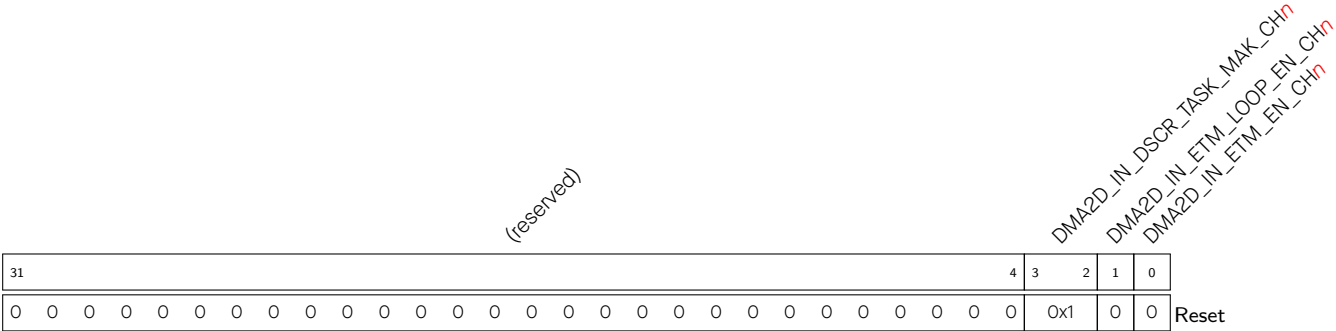
DMA2D_IN_COLOR_PARAM_LO_CHO Configures the coefficient A and B for the least significant byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.30. DMA2D_IN_COLOR_PARAM5_CHO_REG (0x0568)

(reserved)				DMA2D_IN_COLOR_PARAM_L1_CHO																												
31	28	27																													0	
0	0	0	0	Oxbad3000																												Reset

DMA2D_IN_COLOR_PARAM_L1_CHO Configures the coefficient C and D for the least significant byte of the input of color space conversion's second stage for RX channel 0. (R/W)

Register 6.31. DMA2D_IN_ETM_CONF_CH n _REG (n : 0-2) (0x056C+0x100* n)

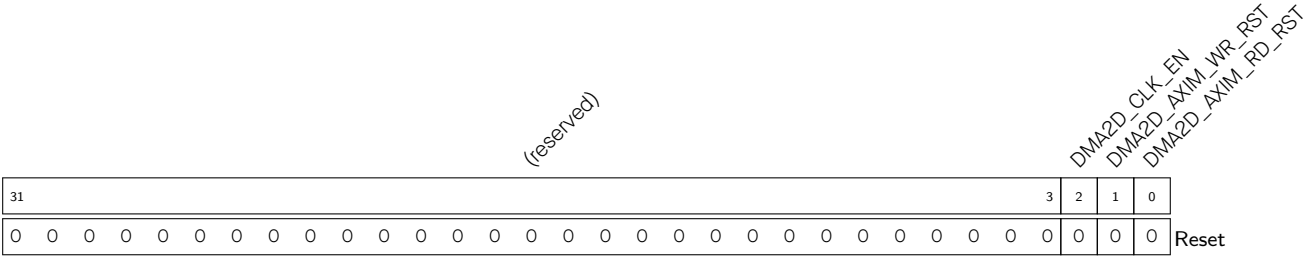


DMA2D_IN_ETM_EN_CH n Configures whether to enable the ETM function for RX channel n .
0: Disable
1: Enable
(R/W)

DMA2D_IN_ETM_LOOP_EN_CH n Configures whether to use the ETM task to indicate the processing of the next descriptor for RX channel n .
0: Not use ETM task
1: Use ETM task
(R/W)

DMA2D_IN_DSCR_TASK_MAK_CH n Configures the maximum number of tasks that can be cached for RX channel n . (R/W)

Register 6.32. DMA2D_RST_CONF_REG (0x0A04)



DMA2D_AXIM_RD_RST Configures whether to reset the AXI master read data FIFO.

- 0: No effect
 - 1: Reset
- (R/W)

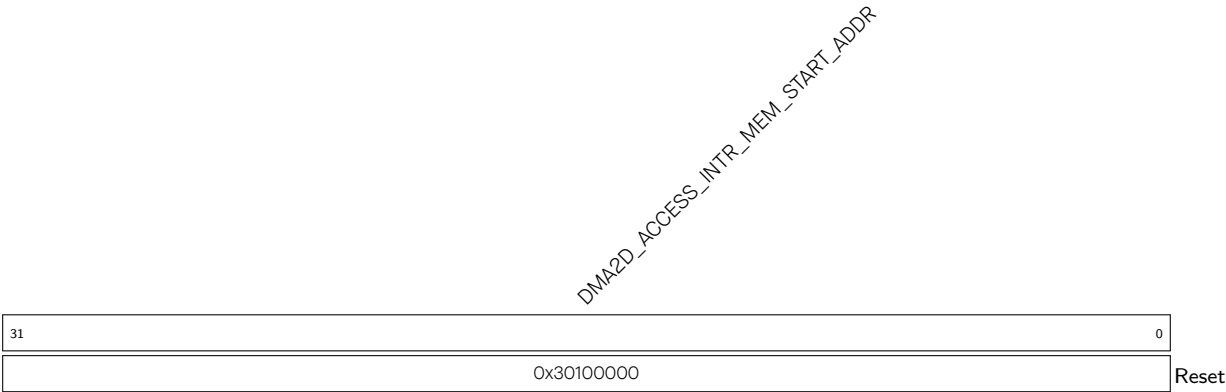
DMA2D_AXIM_WR_RST Configures whether to reset the AXI master write data FIFO.

- 0: No effect
 - 1: Reset
- (R/W)

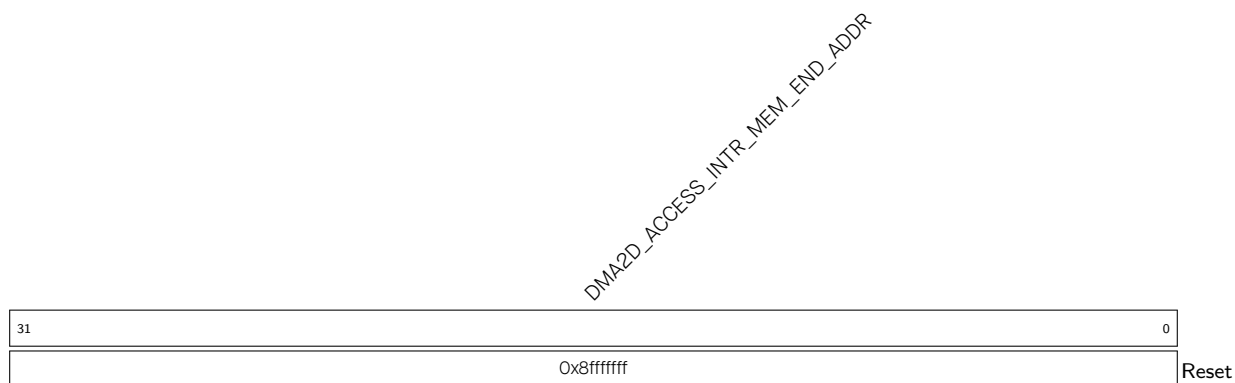
DMA2D_CLK_EN Configures the clock for registers.

- 0: Support clock only when application writes registers
 - 1: Force clock on for register
- (R/W)

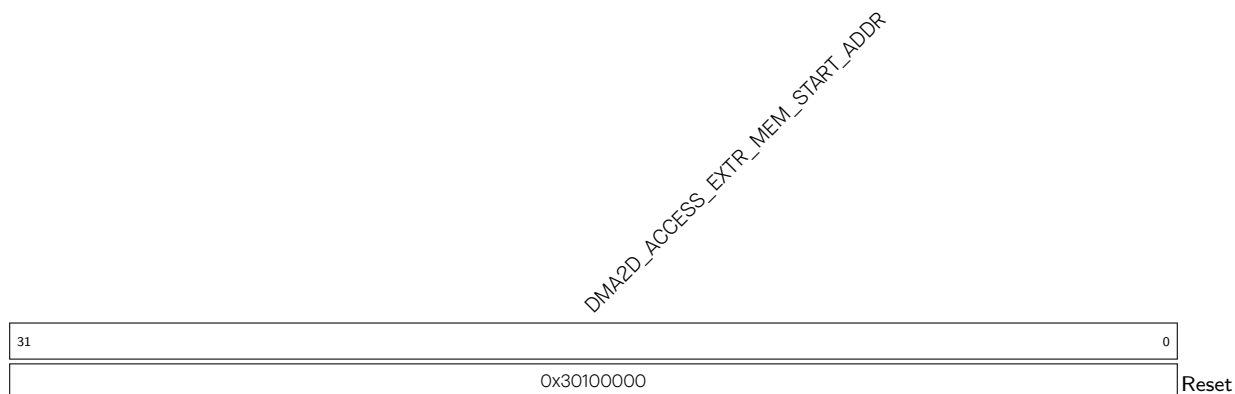
Register 6.33. DMA2D_INTR_MEM_START_ADDR_REG (0x0A08)



DMA2D_ACCESS_INTR_MEM_START_ADDR Configures the start address of the accessible internal address space. (R/W)

Register 6.34. DMA2D_INTR_MEM_END_ADDR_REG (0x0A0C)

DMA2D_ACCESS_INTR_MEM_END_ADDR Configures the end address of the accessible internal address space. Accessing an address beyond this range will result in a descriptor error. (R/W)

Register 6.35. DMA2D_EXTR_MEM_START_ADDR_REG (0x0A10)

DMA2D_ACCESS_EXTR_MEM_START_ADDR Configures the start address of the accessible external address space. (R/W)

Register 6.36. DMA2D_EXTR_MEM_END_ADDR_REG (0x0A14)

Diagram illustrating the structure of the DMA2D_ACCESS_EXTR_MEM_END_ADDR register. The register is 32 bits wide, with bit 31 on the left and bit 0 on the right. The value 0x8ffffff is shown in the center. A label 'Reset' is at the bottom right.

DMA2D_ACCESS_EXTR_MEM_END_ADDR Configures the end address of the accessible external address space. Accessing an address beyond this range will result in a descriptor error. (R/W)

Register 6.37. DMA2D_OUT_ARB_CONFIG_REG (0x0A18)

(reserved)																DMA2D_OUT_WEIGHT_EN		DMA2D_OUT_ARB_TIMEOUT_NUM																			
31																17		16		15																0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0		0x00																Reset			

DMA2D_OUT_ARB_TIMEOUT_NUM Configures the time slot for TX. Measurement unit: AXI bus clock cycle. (R/W)

DMA2D_OUT_WEIGHT_EN Configures whether to enable weight arbitration for TX.

0: Disable

1: Enable

(R/W)

Register 6.38. DMA2D_IN_ARB_CONFIG_REG (0x0A1C)

(reserved)																DMA2D_IN_WEIGHT_EN																DMA2D_IN_ARB_TIMEOUT_NUM															
31																17	16	15																0													
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x00																														
																	Reset																														

DMA2D_IN_ARB_TIMEOUT_NUM Configures the time slot for RX. Measurement unit: AXI bus clock cycle. (R/W)

DMA2D_IN_WEIGHT_EN Configures whether to enable weight arbitration for RX.

0: Disable

1: Enable

(R/W)

Register 6.39. DMA2D_OUT_INT_RAW_CH n _REG (n : 0-3) (0x0004+0x100* n)

(reserved)																												DMA2D_OUT_DSCR_TASK_OVF_CH _n _INT_RAW DMA2D_OUTFIFO_RO_UDF_CHO_INT_RAW DMA2D_OUTFIFO_UDF_L3_CH _n _INT_RAW DMA2D_OUTFIFO_OVF_L3_CH _n _INT_RAW DMA2D_OUTFIFO_UDF_L2_CH _n _INT_RAW DMA2D_OUTFIFO_OVF_L2_CH _n _INT_RAW DMA2D_OUTFIFO_UDF_L1_CH _n _INT_RAW DMA2D_OUTFIFO_OVF_L1_CH _n _INT_RAW DMA2D_OUT_TOTAL_EOF_CH _n _INT_RAW DMA2D_OUT_DSCR_ERR_CH _n _INT_RAW DMA2D_OUT_DONE_CH _n _INT_RAW													
31																		13										12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0													

DMA2D_OUT_DONE_CH n _INT_RAW The raw interrupt status bit of DMA2D_OUT_DONE_CH n _INT. (R/WTC/SS)

DMA2D_OUT_EOF_CH n _INT_RAW The raw interrupt status of DMA2D_OUT_EOF_CH n _INT. (R/WTC/SS)

DMA2D_OUT_DSCR_ERR_CH n _INT_RAW The raw interrupt status of DMA2D_OUT_DSCR_ERR_CH n _INT. (R/WTC/SS)

DMA2D_OUT_TOTAL_EOF_CH n _INT_RAW The raw interrupt status of DMA2D_OUT_TOTAL_EOF_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_OVF_L1_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_OVF_L1_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_UDF_L1_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_UDF_L1_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_OVF_L2_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_OVF_L2_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_UDF_L2_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_UDF_L2_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_OVF_L3_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_OVF_L3_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_UDF_L3_CH n _INT_RAW The raw interrupt status of DMA2D_OUTFIFO_UDF_L3_CH n _INT. (R/WTC/SS)

DMA2D_OUTFIFO_RO_OVF_CHO_INT_RAW The raw interrupt status of DMA2D_OUTFIFO_RO_OVF_CHO_INT. (R/WTC/SS)

DMA2D_OUTFIFO_RO_UDF_CHO_INT_RAW The raw interrupt status of DMA2D_OUTFIFO_RO_UDF_CHO_INT. (R/WTC/SS)

DMA2D_OUT_DSCR_TASK_OVF_CH n _INT_RAW The raw interrupt status of DMA2D_OUT_DSCR_TASK_OVF_CH n _INT. (R/WTC/SS)

Register 6.40. DMA2D_OUT_INT_ENA_CH n _REG (n : 0-3) (0x0008+0x100* n)

(reserved)																								DMA2D_OUT_DSCR_TASK_OVF_CH _n _INT_ENA DMA2D_OUTFIFO_RO_UDF_CHO_INT_ENA DMA2D_OUTFIFO_UDF_L3_CH _n _INT_ENA DMA2D_OUTFIFO_OVF_L3_CH _n _INT_ENA DMA2D_OUTFIFO_UDF_L2_CH _n _INT_ENA DMA2D_OUTFIFO_OVF_L2_CH _n _INT_ENA DMA2D_OUTFIFO_UDF_L1_CH _n _INT_ENA DMA2D_OUTFIFO_OVF_L1_CH _n _INT_ENA DMA2D_OUT_TOTAL_EOF_CH _n _INT_ENA DMA2D_OUT_DSCR_ERR_CH _n _INT_ENA DMA2D_OUT_DONE_CH _n _INT_ENA													
31												13												12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0										

DMA2D_OUT_DONE_CH n _INT_ENA Write 1 to enable the DMA2D_OUT_DONE_CH n _INT. (R/W)

DMA2D_OUT_EOF_CH n _INT_ENA Write 1 to enable the DMA2D_OUT_EOF_CH n _INT. (R/W)

DMA2D_OUT_DSCR_ERR_CH n _INT_ENA Write 1 to enable the DMA2D_OUT_DSCR_ERR_CH n _INT. (R/W)

DMA2D_OUT_TOTAL_EOF_CH n _INT_ENA Write 1 to enable the DMA2D_OUT_TOTAL_EOF_CH n _INT. (R/W)

DMA2D_OUTFIFO_OVF_L1_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_OVF_L1_CH n _INT. (R/W)

DMA2D_OUTFIFO_UDF_L1_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_UDF_L1_CH n _INT. (R/W)

DMA2D_OUTFIFO_OVF_L2_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_OVF_L2_CH n _INT. (R/W)

DMA2D_OUTFIFO_UDF_L2_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_UDF_L2_CH n _INT. (R/W)

DMA2D_OUTFIFO_OVF_L3_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_OVF_L3_CH n _INT. (R/W)

DMA2D_OUTFIFO_UDF_L3_CH n _INT_ENA Write 1 to enable the DMA2D_OUTFIFO_UDF_L3_CH n _INT. (R/W)

DMA2D_OUTFIFO_RO_OVF_CHO_INT_ENA Write 1 to enable the DMA2D_OUTFIFO_RO_OVF_CHO_INT. (R/W)

DMA2D_OUTFIFO_RO_UDF_CHO_INT_ENA Write 1 to enable the DMA2D_OUTFIFO_RO_UDF_CHO_INT. (R/W)

DMA2D_OUT_DSCR_TASK_OVF_CH n _INT_ENA Write 1 to enable the DMA2D_OUT_DSCR_TASK_OVF_CH n _INT. (R/W)

Register 6.41. DMA2D_OUT_INT_ST_CH n _REG (n : 0-3) (0x000C+0x100* n)

(reserved)																								DMA2D_OUT_DSCR_TASK_OVF_CH _n _INT_ST DMA2D_OUTFIFO_RO_UDF_CHO_INT_ST DMA2D_OUTFIFO_UDF_L3_CH _n _INT_ST DMA2D_OUTFIFO_OVF_L3_CH _n _INT_ST DMA2D_OUTFIFO_UDF_L2_CH _n _INT_ST DMA2D_OUTFIFO_OVF_L2_CH _n _INT_ST DMA2D_OUTFIFO_UDF_L1_CH _n _INT_ST DMA2D_OUTFIFO_OVF_L1_CH _n _INT_ST DMA2D_OUT_TOTAL_EOF_CH _n _INT_ST DMA2D_OUT_DSCR_ERR_CH _n _INT_ST DMA2D_OUT_DONE_CH _n _INT_ST													
31																13								12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0											

DMA2D_OUT_DONE_CH n _INT_ST The masked interrupt status of DMA2D_OUT_DONE_CH n _INT. (RO)

DMA2D_OUT_EOF_CH n _INT_ST The masked interrupt status of DMA2D_OUT_EOF_CH n _INT. (RO)

DMA2D_OUT_DSCR_ERR_CH n _INT_ST The masked interrupt status of DMA2D_OUT_DSCR_ERR_CH n _INT. (RO)

DMA2D_OUT_TOTAL_EOF_CH n _INT_ST The masked interrupt status of DMA2D_OUT_TOTAL_EOF_CH n _INT. (RO)

DMA2D_OUTFIFO_OVF_L1_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_OVF_L1_CH n _INT. (RO)

DMA2D_OUTFIFO_UDF_L1_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_UDF_L1_CH n _INT. (RO)

DMA2D_OUTFIFO_OVF_L2_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_OVF_L2_CH n _INT. (RO)

DMA2D_OUTFIFO_UDF_L2_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_UDF_L2_CH n _INT. (RO)

DMA2D_OUTFIFO_OVF_L3_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_OVF_L3_CH n _INT. (RO)

DMA2D_OUTFIFO_UDF_L3_CH n _INT_ST The masked interrupt status of DMA2D_OUTFIFO_UDF_L3_CH n _INT. (RO)

DMA2D_OUTFIFO_RO_OVF_CHO_INT_ST The masked interrupt status of DMA2D_OUTFIFO_RO_OVF_CHO_INT. (RO)

DMA2D_OUTFIFO_RO_UDF_CHO_INT_ST The masked interrupt status of DMA2D_OUTFIFO_RO_UDF_CHO_INT. (RO)

DMA2D_OUT_DSCR_TASK_OVF_CH n _INT_ST The masked interrupt status of DMA2D_OUT_DSCR_TASK_OVF_CH n _INT. (RO)

Register 6.42. DMA2D_OUT_INT_CLR_CH n _REG (n : 0-3) (0x0010+0x100* n)

(reserved)																								DMA2D_OUT_DSCR_TASK_OVF_CH _n _INT_CLR DMA2D_OUTFIFO_RO_UDF_CHO_INT_CLR DMA2D_OUTFIFO_UDF_L3_CH _n _INT_CLR DMA2D_OUTFIFO_OVF_L3_CH _n _INT_CLR DMA2D_OUTFIFO_UDF_L2_CH _n _INT_CLR DMA2D_OUTFIFO_OVF_L2_CH _n _INT_CLR DMA2D_OUTFIFO_UDF_L1_CH _n _INT_CLR DMA2D_OUTFIFO_OVF_L1_CH _n _INT_CLR DMA2D_OUT_TOTAL_EOF_CH _n _INT_CLR DMA2D_OUT_DSCR_ERR_CH _n _INT_CLR DMA2D_OUT_EOF_DONE_CH _n _INT_CLR													
31												13												12	11	10	9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0										

DMA2D_OUT_DONE_CH n _INT_CLR Write 1 to clear the DMA2D_OUT_DONE_CH n _INT. (WT)

DMA2D_OUT_EOF_CH n _INT_CLR Write 1 to clear the DMA2D_OUT_EOF_CH n _INT. (WT)

DMA2D_OUT_DSCR_ERR_CH n _INT_CLR Write 1 to clear the DMA2D_OUT_DSCR_ERR_CH n _INT.
(WT)

DMA2D_OUT_TOTAL_EOF_CH n _INT_CLR Write 1 to clear the DMA2D_OUT_TOTAL_EOF_CH n _INT.
(WT)

DMA2D_OUTFIFO_OVF_L1_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_OVF_L1_CH n _INT.
(WT)

DMA2D_OUTFIFO_UDF_L1_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_UDF_L1_CH n _INT.
(WT)

DMA2D_OUTFIFO_OVF_L2_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_OVF_L2_CH n _INT.
(WT)

DMA2D_OUTFIFO_UDF_L2_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_UDF_L2_CH n _INT.
(WT)

DMA2D_OUTFIFO_OVF_L3_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_OVF_L3_CH n _INT.
(WT)

DMA2D_OUTFIFO_UDF_L3_CH n _INT_CLR Write 1 to clear the DMA2D_OUTFIFO_UDF_L3_CH n _INT.
(WT)

DMA2D_OUTFIFO_RO_OVF_CHO_INT_CLR Write 1 to clear the DMA2D_OUTFIFO_RO_OVF_CHO_INT.
(WT)

DMA2D_OUTFIFO_RO_UDF_CHO_INT_CLR Write 1 to clear the DMA2D_OUTFIFO_RO_UDF_CHO_INT.
(WT)

DMA2D_OUT_DSCR_TASK_OVF_CH n _INT_CLR Write 1 to clear the DMA2D_OUT_DSCR_TASK_OVF_CH n _INT. (WT)

Register 6.43. DMA2D_IN_INT_RAW_CH n _REG (n : 0-2) (0x0504+0x100* n)

(reserved)																												DMA2D_IN_DSCR_TASK_OVF_CH _n _INT_RAW DMA2D_INFIFO_RO_UDF_CHO_INT_RAW DMA2D_INFIFO_UDF_L3_CH _n _INT_RAW DMA2D_INFIFO_OVF_L3_CH _n _INT_RAW DMA2D_INFIFO_UDF_L2_CH _n _INT_RAW DMA2D_INFIFO_OVF_L2_CH _n _INT_RAW DMA2D_INFIFO_UDF_L1_CH _n _INT_RAW DMA2D_IN_DSCR_ERR_CH _n _INT_RAW DMA2D_IN_SUC_EOF_CH _n _INT_RAW DMA2D_IN_DONE_CH _n _INT_RAW															
31																14												13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0																0												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

DMA2D_IN_DONE_CH n _INT_RAW The raw interrupt status of DMA2D_IN_DONE_CH n _INT. (R/WTC/SS)

DMA2D_IN_SUC_EOF_CH n _INT_RAW The raw interrupt status of DMA2D_IN_SUC_EOF_CH n _INT. (R/WTC/SS)

DMA2D_IN_ERR_EOF_CH n _INT_RAW The raw interrupt status of DMA2D_IN_ERR_EOF_CH n _INT. (R/WTC/SS)

DMA2D_IN_DSCR_ERR_CH n _INT_RAW The raw interrupt status of DMA2D_IN_DSCR_ERR_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_OVF_L1_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_OVF_L1_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_UDF_L1_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_UDF_L1_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_OVF_L2_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_OVF_L2_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_UDF_L2_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_UDF_L2_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_OVF_L3_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_OVF_L3_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_UDF_L3_CH n _INT_RAW The raw interrupt status of DMA2D_INFIFO_UDF_L3_CH n _INT. (R/WTC/SS)

DMA2D_IN_DSCR_EMPTY_CH n _INT_RAW The raw interrupt status of DMA2D_IN_DSCR_EMPTY_CH n _INT. (R/WTC/SS)

DMA2D_INFIFO_RO_OVF_CHO_INT_RAW The raw interrupt status of DMA2D_INFIFO_RO_OVF_CHO_INT. (R/WTC/SS)

DMA2D_INFIFO_RO_UDF_CHO_INT_RAW The raw interrupt status of DMA2D_INFIFO_RO_UDF_CHO_INT. (R/WTC/SS)

DMA2D_IN_DSCR_TASK_OVF_CH n _INT_RAW The raw interrupt status of DMA2D_IN_DSCR_TASK_OVF_CH n _INT. (R/WTC/SS)

Register 6.44. DMA2D_IN_INT_ENA_CH_n_REG (*n*: 0-2) (0x0508+0x100**n*)

(reserved)																												DMA2D_IN_DSCR_TASK_OVF_CH _n _INT_ENA DMA2D_INFIFO_RO_UDF_CHO_INT_ENA DMA2D_IN_DSCR_RO_OVF_CHO_INT_ENA DMA2D_IN_DSCR_EMPTY_CH _n _INT_ENA DMA2D_INFIFO_UDF_L3_CH _n _INT_ENA DMA2D_INFIFO_OVF_L3_CH _n _INT_ENA DMA2D_INFIFO_UDF_L2_CH _n _INT_ENA DMA2D_INFIFO_OVF_L2_CH _n _INT_ENA DMA2D_INFIFO_UDF_L1_CH _n _INT_ENA DMA2D_IN_DSCR_ERR_CH _n _INT_ENA DMA2D_IN_ERR_EOF_CH _n _INT_ENA DMA2D_IN_SUC_EOF_CH _n _INT_ENA DMA2D_IN_DONE_CH _n _INT_ENA															
31														14														13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0														0														0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

DMA2D_IN_DONE_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_DONE_CH_n_INT. (R/W)

DMA2D_IN_SUC_EOF_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_SUC_EOF_CH_n_INT. (R/W)

DMA2D_IN_ERR_EOF_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_ERR_EOF_CH_n_INT. (R/W)

DMA2D_IN_DSCR_ERR_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_DSCR_ERR_CH_n_INT. (R/W)

DMA2D_INFIFO_OVF_L1_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_OVF_L1_CH_n_INT. (R/W)

DMA2D_INFIFO_UDF_L1_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_UDF_L1_CH_n_INT. (R/W)

DMA2D_INFIFO_OVF_L2_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_OVF_L2_CH_n_INT. (R/W)

DMA2D_INFIFO_UDF_L2_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_UDF_L2_CH_n_INT. (R/W)

DMA2D_INFIFO_OVF_L3_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_OVF_L3_CH_n_INT. (R/W)

DMA2D_INFIFO_UDF_L3_CH_n_INT_ENA Write 1 to enable the DMA2D_INFIFO_UDF_L3_CH_n_INT. (R/W)

DMA2D_IN_DSCR_EMPTY_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_DSCR_EMPTY_CH_n_INT. (R/W)

DMA2D_INFIFO_RO_OVF_CHO_INT_ENA Write 1 to enable the DMA2D_INFIFO_RO_OVF_CHO_INT. (R/W)

DMA2D_INFIFO_RO_UDF_CHO_INT_ENA Write 1 to enable the DMA2D_INFIFO_RO_UDF_CHO_INT. (R/W)

DMA2D_IN_DSCR_TASK_OVF_CH_n_INT_ENA Write 1 to enable the DMA2D_IN_DSCR_TASK_OVF_CH_n_INT. (R/W)

Register 6.45. DMA2D_IN_INT_ST_CH n _REG (n : 0-2) (0x050C+0x100* n)

(reserved)																												DMA2D_IN_DSCR_TASK_OVF_CH _n _INT_ST DMA2D_INFIFO_RO_OVF_CHO_INT_ST DMA2D_INFIFO_UDF_CHO_INT_ST DMA2D_IN_DSCR_EMPTY_CH _n _INT_ST DMA2D_INFIFO_OVF_L3_CH _n _INT_ST DMA2D_INFIFO_UDF_L3_CH _n _INT_ST DMA2D_INFIFO_OVF_L2_CH _n _INT_ST DMA2D_INFIFO_UDF_L2_CH _n _INT_ST DMA2D_IN_DSCR_ERR_CH _n _INT_ST DMA2D_IN_ERR_EOF_CH _n _INT_ST DMA2D_IN_SUC_EOF_CH _n _INT_ST DMA2D_IN_DONE_CH _n _INT_ST													
31														14														13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset											

DMA2D_IN_DONE_CH n _INT_ST The masked interrupt status of the DMA2D_IN_DONE_CH n _INT. (RO)

DMA2D_IN_SUC_EOF_CH n _INT_ST The masked interrupt status of the DMA2D_IN_SUC_EOF_CH n _INT. (RO)

DMA2D_IN_ERR_EOF_CH n _INT_ST The masked interrupt status of the DMA2D_IN_ERR_EOF_CH n _INT. (RO)

DMA2D_IN_DSCR_ERR_CH n _INT_ST The masked interrupt status of the DMA2D_IN_DSCR_ERR_CH n _INT. (RO)

DMA2D_INFIFO_OVF_L1_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_OVF_L1_CH n _INT. (RO)

DMA2D_INFIFO_UDF_L1_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_UDF_L1_CH n _INT. (RO)

DMA2D_INFIFO_OVF_L2_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_OVF_L2_CH n _INT. (RO)

DMA2D_INFIFO_UDF_L2_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_UDF_L2_CH n _INT. (RO)

DMA2D_INFIFO_OVF_L3_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_OVF_L3_CH n _INT. (RO)

DMA2D_INFIFO_UDF_L3_CH n _INT_ST The masked interrupt status of the DMA2D_INFIFO_UDF_L3_CH n _INT. (RO)

DMA2D_IN_DSCR_EMPTY_CH n _INT_ST The masked interrupt status of the DMA2D_IN_DSCR_EMPTY_CH n _INT. (RO)

DMA2D_INFIFO_RO_OVF_CHO_INT_ST The masked interrupt status of the DMA2D_INFIFO_RO_OVF_CHO_INT. (RO)

DMA2D_INFIFO_RO_UDF_CHO_INT_ST The masked interrupt status of the DMA2D_INFIFO_RO_UDF_CHO_INT. (RO)

DMA2D_IN_DSCR_TASK_OVF_CH n _INT_ST The masked interrupt status of the DMA2D_IN_DSCR_TASK_OVF_CH n _INT. (RO)

Register 6.46. DMA2D_IN_INT_CLR_CH_n_REG (*n*: 0-2) (0x0510+0x100**n*)

(reserved)																DMA2D_IN_DSCR_TASK_OVF_CH _n _INT_CLR DMA2D_IN_FIFO_RO_UDF_CHO_INT_CLR DMA2D_IN_DSCR_EMPTY_CH _n _INT_CLR DMA2D_IN_FIFO_UDF_L3_CH _n _INT_CLR DMA2D_IN_FIFO_OVF_L3_CH _n _INT_CLR DMA2D_IN_FIFO_UDF_L2_CH _n _INT_CLR DMA2D_IN_FIFO_OVF_L2_CH _n _INT_CLR DMA2D_IN_DSCR_ERR_CH _n _INT_CLR DMA2D_IN_ERR_EOF_CH _n _INT_CLR DMA2D_IN_SUC_EOF_CH _n _INT_CLR DMA2D_IN_DONE_CH _n _INT_CLR															
31																14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

DMA2D_IN_DONE_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_DONE_CH_n_INT. (WT)

DMA2D_IN_SUC_EOF_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_SUC_EOF_CH_n_INT. (WT)

DMA2D_IN_ERR_EOF_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_ERR_EOF_CH_n_INT. (WT)

DMA2D_IN_DSCR_ERR_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_DSCR_ERR_CH_n_INT. (WT)

DMA2D_IN_FIFO_OVF_L1_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_OVF_L1_CH_n_INT. (WT)

DMA2D_IN_FIFO_UDF_L1_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_UDF_L1_CH_n_INT. (WT)

DMA2D_IN_FIFO_OVF_L2_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_OVF_L2_CH_n_INT. (WT)

DMA2D_IN_FIFO_UDF_L2_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_UDF_L2_CH_n_INT. (WT)

DMA2D_IN_FIFO_OVF_L3_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_OVF_L3_CH_n_INT. (WT)

DMA2D_IN_FIFO_UDF_L3_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_UDF_L3_CH_n_INT. (WT)

DMA2D_IN_DSCR_EMPTY_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_DSCR_EMPTY_CH_n_INT. (WT)

DMA2D_IN_FIFO_RO_OVF_CHO_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_RO_OVF_CHO_INT. (WT)

DMA2D_IN_FIFO_RO_UDF_CHO_INT_CLR Write 1 to clear the DMA2D_IN_FIFO_RO_UDF_CHO_INT. (WT)

DMA2D_IN_DSCR_TASK_OVF_CH_n_INT_CLR Write 1 to clear the DMA2D_IN_DSCR_TASK_OVF_CH_n_INT. (WT)

Register 6.47. DMA2D_OUTFIFO_STATUS_CH n _REG (n : 0-3) (0x0014+0x100* n)

(reserved)			DMA2D_OUTFIFO_CNT_L3_CHn			DMA2D_OUTFIFO_EMPTY_L3_CHn			DMA2D_OUTFIFO_FULL_L3_CHn			DMA2D_OUTFIFO_CNT_L1_CHn			DMA2D_OUTFIFO_EMPTY_L1_CHn			DMA2D_OUTFIFO_FULL_L1_CHn			DMA2D_OUTFIFO_EMPTY_L2_CHn			DMA2D_OUTFIFO_FULL_L2_CHn		
31	29	28	24	23	22	21	17	16	15	14	13	12	11	10	9	8	7	6	5	2	1	0				
0	0	0	Ox0	1	0	Ox0	1	0	1	1	1	1	1	1	1	1	1	0	Ox0	1	0	Reset				

DMA2D_OUTFIFO_FULL_L2_CH n Represents whether the L2 TX FIFO for TX channel n is full.

0: Not full

1: Full

(RO)

DMA2D_OUTFIFO_EMPTY_L2_CH n Represents whether the L2 TX FIFO for TX channel n is empty.

0: Not empty

1: Empty

(RO)

DMA2D_OUTFIFO_CNT_L2_CH n Represents the number of data bytes in L2 TX FIFO for TX channel n . (RO)

DMA2D_OUT_REMAIN_UNDER_1B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_2B_CH_n Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_3B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_4B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_5B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_6B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_7B_CHn Reserved. (RO)

DMA2D_OUT_REMAIN_UNDER_8B_CHn Reserved. (RO)

DMA2D_OUTFIFO_FULL_L1_CH n Represents whether the L1 TX FIFO for TX channel n is full.

0: Not full

1: Full

(RO)

DMA2D_OUTFIFO_EMPTY_L1_CH n Represents whether the L1 TX FIFO for TX channel n is empty.

0: Not empty

1: Empty

(RO)

DMA2D_OUTFIFO_CNT_L1_CH n Represents the number of data bytes in L1 TX FIFO for TX channel n . (RO)

Continued on the next page...

Register 6.47. DMA2D_OUTFIFO_STATUS_CHn_REG (n: 0-3) (0x0014+0x100*n)

Continued from the previous page...

DMA2D_OUTFIFO_FULL_L3_CHn Represents whether the L3 TX FIFO for TX channel n is full.

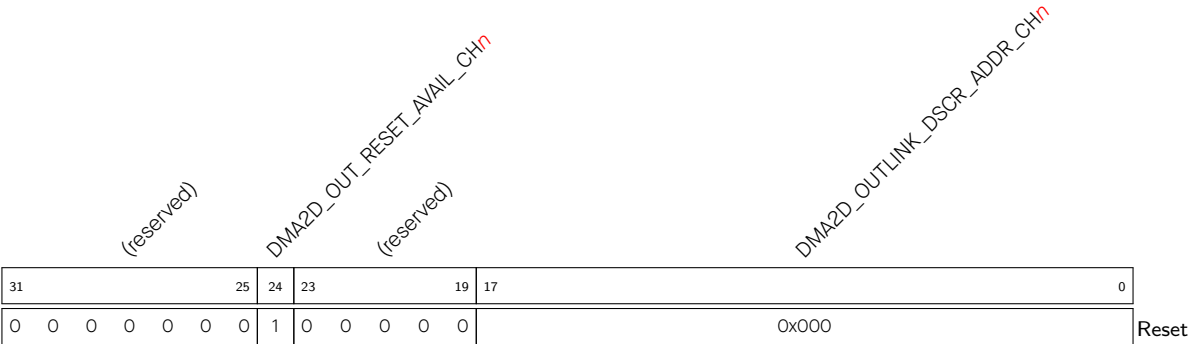
- 0: Not full
 - 1: Full
- (RO)

DMA2D_OUTFIFO_EMPTY_L3_CHn Represents whether the L3 TX FIFO for TX channel n is empty.

- 0: Not empty
 - 1: Empty
- (RO)

DMA2D_OUTFIFO_CNT_L3_CHn Represents the number of data bytes in L3 TX FIFO for TX channel n. (RO)

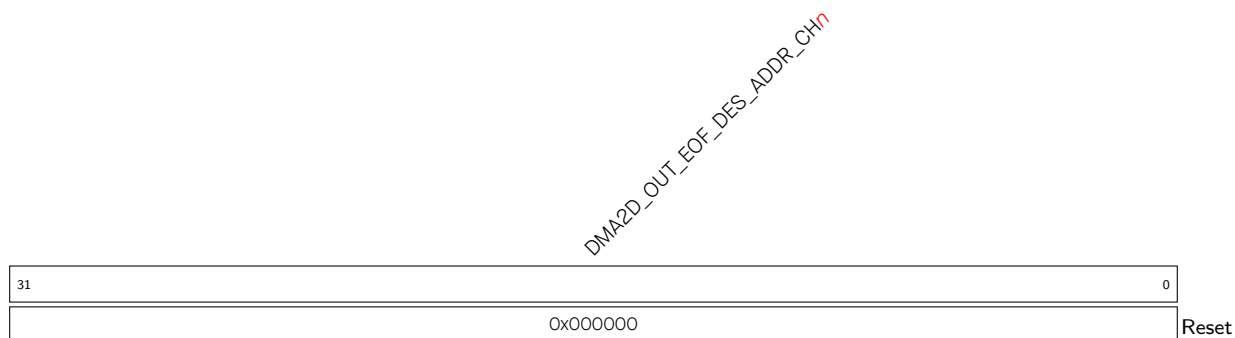
Register 6.48. DMA2D_OUT_STATE_CHn_REG (n: 0-3) (0x0024+0x100*n)



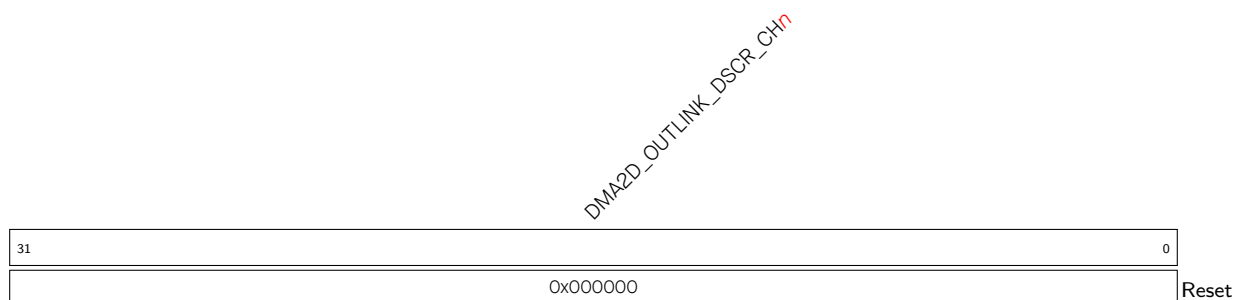
DMA2D_OUTLINK_DSCR_ADDR_CHn Represents the lower 18 bits of the next transmit descriptor address that is pre-read (but not processed yet). If the current transmit descriptor is the last descriptor, then this field represents the address of the current transmit descriptor. (RO)

DMA2D_OUT_RESET_AVAIL_CHn Represents whether it is safe to reset TX channel n.

- 0: Not safe
 - 1: Safe
- (RO)

Register 6.49. DMA2D_OUT_EOF_DES_ADDR_CH n _REG (n : 0-3) (0x0028+0x100* n)

DMA2D_OUT_EOF_DES_ADDR_CH n Represents the address of the transmit descriptor when the eof bit in this descriptor is 1. (RO)

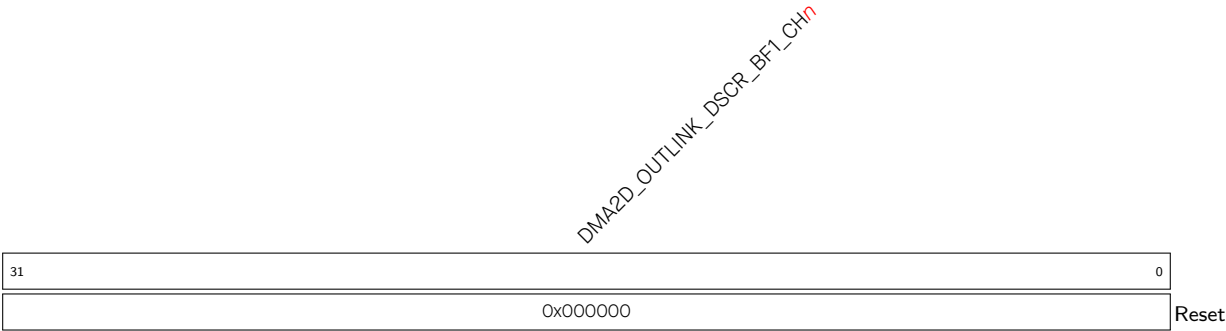
Register 6.50. DMA2D_OUT_DSCR_CH n _REG (n : 0-3) (0x002C+0x100* n)

DMA2D_OUTLINK_DSCR_CH n Represents the address of the next transmit descriptor pointed by the current transmit descriptor that is pre-read. (RO)

Register 6.51. DMA2D_OUT_DSCR_BFO_CH n _REG (n : 0-3) (0x0030+0x100* n)

DMA2D_OUTLINK_DSCR_BFO_CH n Represents the address of the current transmit descriptor that is pre-read. (RO)

Register 6.52. DMA2D_OUT_DSCR_BF1_CHn_REG (n: 0-3) (0x0034+0x100*n)



DMA2D_OUTLINK_DSCR_BF1_CHn Represents the address of the previous transmit descriptor that is pre-read. (RO)

Register 6.53. DMA2D_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0514+0x100**n*)

(reserved)			DMA2D_INFIFO_CNT_L3_CH ⁿ				DMA2D_INFIFO_EMPTY_L3_CH ⁿ				DMA2D_INFIFO_FULL_L3_CH ⁿ				DMA2D_INFIFO_CNT_L1_CH ⁿ				DMA2D_INFIFO_EMPTY_L1_CH ⁿ				DMA2D_IN_REMAIN_UNDER_8B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_7B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_6B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_5B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_4B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_3B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_2B_CH ⁿ				DMA2D_IN_REMAIN_UNDER_1B_CH ⁿ				(reserved)				DMA2D_INFIFO_CNT_L2_CH ⁿ				DMA2D_INFIFO_EMPTY_L2_CH ⁿ				DMA2D_INFIFO_FULL_L2_CH ⁿ			
31	29	28	24	23	22	21	17	16	15	14	13	12	11	10	9	8	7	6	5	2	1	0																																																
0	0	0	0x0	1	0	0x0	1	0	0	0	0	0	0	0	0	0	0	0	0	0x0	1	0	Reset																																															

DMA2D_INFIFO_FULL_L2_CH n Represents whether the L2 RX FIFO for RX channel n is full.

0: Not full

1: Full

(RO)

DMA2D_INFIFO_EMPTY_L2_CH n Represents whether the L2 RX FIFO for RX channel n is empty.

0: Not empty

1: Empty

(RO)

DMA2D_INFIFO_CNT_L2_CH n Represents the number of data bytes in the L2 RX FIFO for RX channel n . (RO)

DMA2D_IN_REMAIN_UNDER_1B_CHn Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_2B_CH_n Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_3B_CHn Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_4B_CH_n Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_5B_CHn Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_6B_CHn Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_7B_CH_n Reserved. (RO)

DMA2D_IN_REMAIN_UNDER_8B_CHn Reserved. (RO)

DMA2D_INFIFO_FULL_L1_CH n Represents whether the L1 RX FIFO for RX channel n is full.

0: Not full

1: Full

(RO)

DMA2D_INFIFO_EMPTY_L1_CH n Represents whether the L1 RX FIFO for RX channel n is empty.

0: Not empty

1: Empty

(RO)

DMA2D_INFIFO_CNT_L1_CH n Represents the number of data bytes in the L1 RX FIFO for RX channel n . (RO)

Continued on the next page...

Register 6.53. DMA2D_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0514+0x100**n*)

Continued from the previous page...

DMA2D_INFIFO_FULL_L3_CH n Represents whether the L3 RX FIFO for RX channel n is full.

0: Not full

1: Full

(RO)

DMA2D_INFIFO_EMPTY_L3_CH n Represents whether the L3 RX FIFO for RX channel n is empty.

0: Not empty

1: Empty

(RO)

DMA2D_INFIFO_CNT_L3_CH n Represents the number of data bytes in the L3 RX FIFO for RX channel n . (RO)

Register 6.54. DMA2D_IN_STATE_CH n _REG (n : 0-2) (0x0524+0x100* n)

(reserved)								DMA2D_IN_RESET_AVAIL_CH ⁿ					(reserved)														DMA2D_INLINK_DSCR_ADDR_CH ⁿ																										
31								24								23					19					17														0													
0 0 0 0 0 0 0 0								1								0 0 0 0 0					0x000														Reset																		

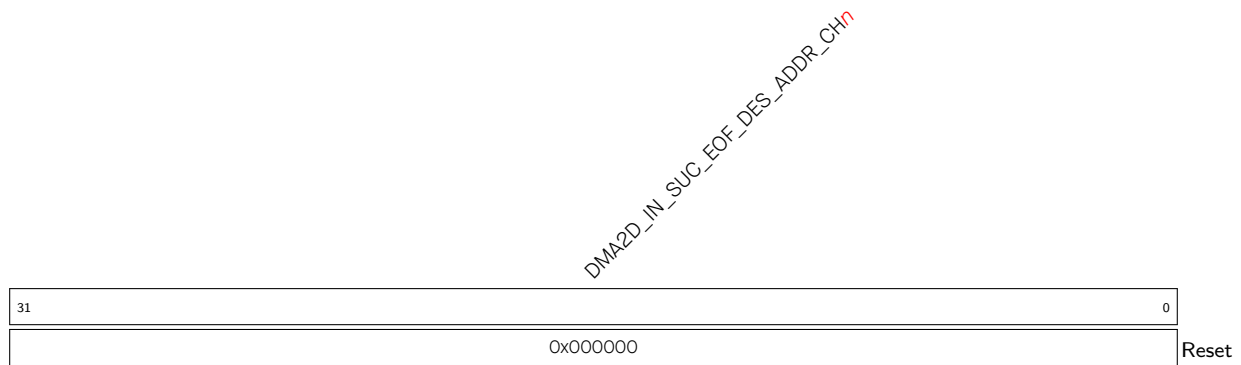
DMA2D_INLINK_DSCR_ADDR_CH_n Represents the lower 18 bits of the next receive descriptor address that is pre-read (but not processed yet). If the current receive descriptor is the last descriptor, then this field represents the address of the current receive descriptor. (RO)

DMA2D_IN_RESET_AVAIL_CH n Represents whether it is safe to reset RX channel n .

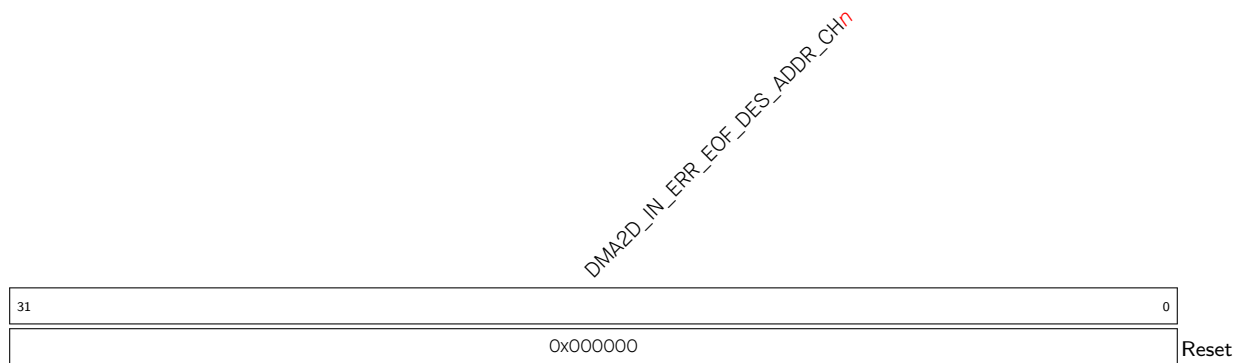
0: Not safe

1: Safe

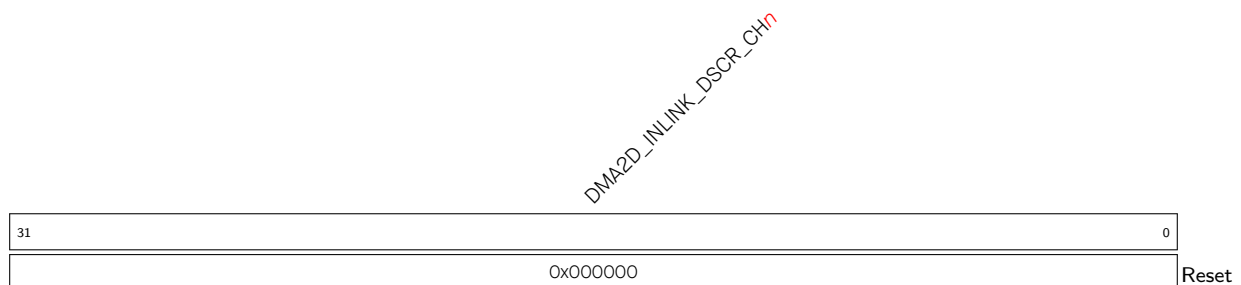
(RO)

Register 6.55. DMA2D_IN_SUC_EOF_DES_ADDR_CH n _REG (n : 0-2) (0x0528+0x100* n)

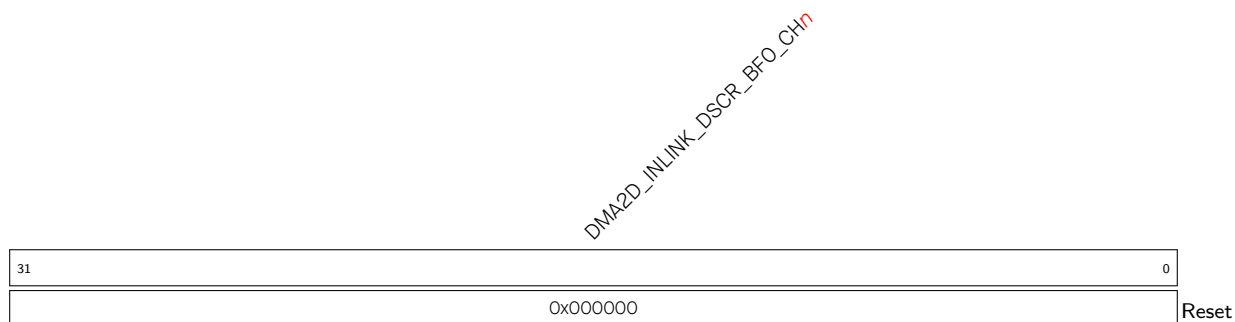
DMA2D_IN_SUC_EOF_DES_ADDR_CH n Represents the address of the receive descriptor when the eof bit in this descriptor is 1. (RO)

Register 6.56. DMA2D_IN_ERR_EOF_DES_ADDR_CH n _REG (n : 0-2) (0x052C+0x100* n)

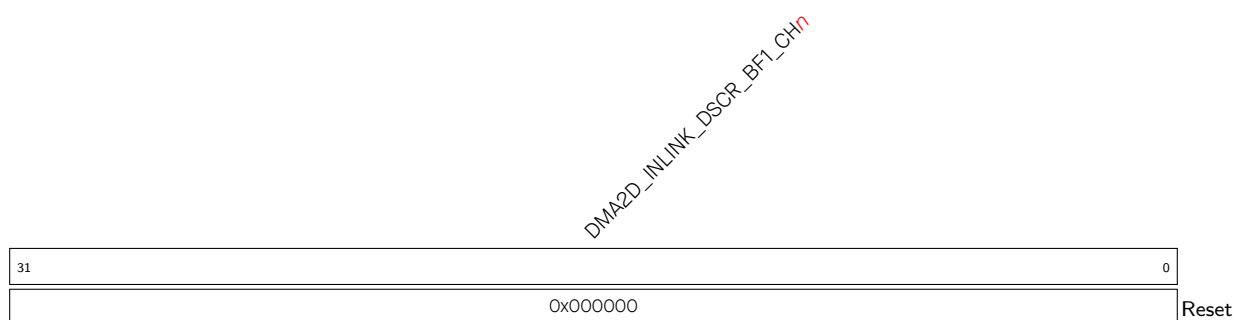
DMA2D_IN_ERR_EOF_DES_ADDR_CH n Represents the address of the receive descriptor when there are some errors in the currently received data. Valid only for JPEG. (RO)

Register 6.57. DMA2D_IN_DSCR_CH n _REG (n : 0-2) (0x0530+0x100* n)

DMA2D_INLINK_DSCR_CH n Represents the address of the next receive descriptor pointed by the current receive descriptor that is pre-read. (RO)

Register 6.58. DMA2D_IN_DSCR_BFO_CH n _REG (n : 0-2) (0x0534+0x100* n)

DMA2D_INLINK_DSCR_BFO_CH n Represents the address of the current receive descriptor that is pre-read. (RO)

Register 6.59. DMA2D_IN_DSCR_BF1_CH n _REG (n : 0-2) (0x0538+0x100* n)

DMA2D_INLINK_DSCR_BF1_CH n Represents the address of the previous receive descriptor that is pre-read. (RO)

Register 6.60. DMA2D_AXI_ERR_REG (0x0A00)

[illegible]

DMA2D_RID_ERR_CNT Represents the number of AXI read ID errors. (RO)

DMA2D_RRESP_ERR_CNT Represents the number of AXI read response errors. (RO)

DMA2D_WRESP_ERR_CNT Represents the number of AXI write response errors. (RO)

DMA2D_RD_FIFO_CNT	Represents the number of remaining commands in the AXI read command FIFO. (RO)
--------------------------	--

DMA2D_RD_BAK_FIFO_CNT Represents the number of remaining commands in the AXI read backup command FIFO. (RO)

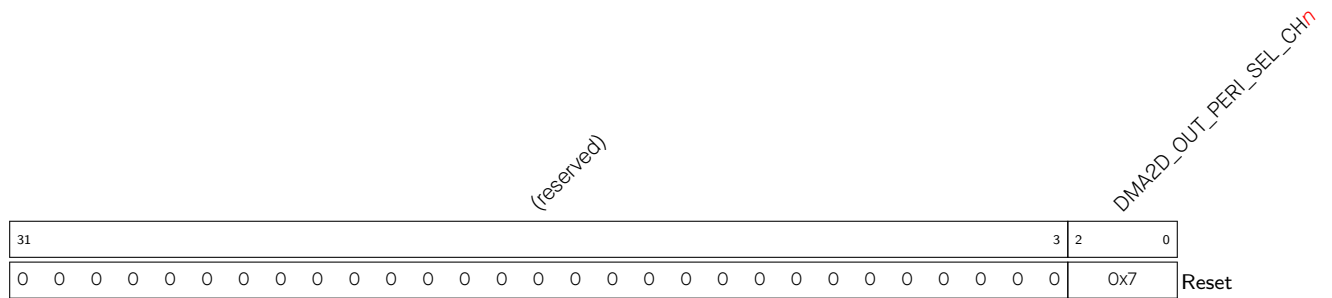
DMA2D_WR_FIFO_CNT	Represents the number of remaining commands in the AXI write command FIFO. (RO)
--------------------------	---

DMA2D_WR_BAK_FIFO_CNT Represents the number of remaining commands in the AXI write backup command FIFO. (RO)

Register 6.61. DMA2D_DATE_REG (0x0A2C)

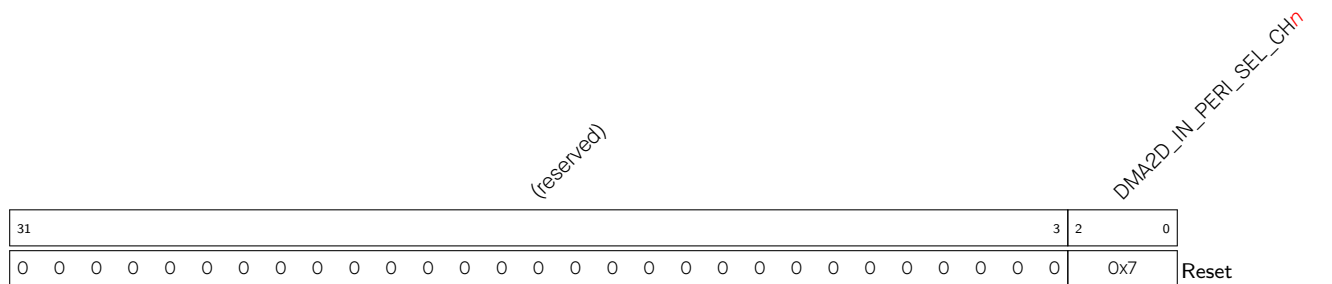
Diagram of DMA2D control register DMA2D_CR. The register is 32 bits wide. Bit 31 is labeled 'DMA2D_CR'. Bit 0 is labeled 'Reset'. The value 0x2412190 is shown in the register.

DMA2D_DATE Version control register. (R/W)

Register 6.62. DMA2D_OUT_PERI_SEL_CH n _REG (n : 0-3) (0x0038+0x100* n)

DMA2D_OUT_PERI_SEL_CH n Configures the peripheral connected to TX channel n .

- 0: JPEG
 - 1: SRM module of PPA
 - 2: BLEND0 channel of PPA
 - 3: BLEND1 channel of PPA
 - 4 ~ 7: Dummy 4 ~ 7
- (R/W)

Register 6.63. DMA2D_IN_PERI_SEL_CH n _REG (n : 0-2) (0x053C+0x100* n)

DMA2D_IN_PERI_SEL_CH n Configures the peripheral connected to RX channel n .

- 0: JPEG
 - 1: SRM module of PPA
 - 2: BLEND module of PPA
 - 3 ~ 7: Dummy 3 ~ 7
- (R/W)